第0章: 導入と全体像 — このアプリは何で、どう動くのか（完全版）

> この章は、これからこの「業務工数（こうすう）システム」を引き継ぐあなたが、**最初の1日目に読む地図** です。まだ1行もコードを書く必要はありません。まずは「このアプリは何のために作られたのか」「ブラウザに表示された画面の裏で、いったい何が起きているのか」「ファイルがどこに何のために置いてあるのか」を、**実際のソースコードを1行ずつ追いながら** つかんでいきます。
>
> 特にこの章の山場は、**ログインボタンを押してから、画面が次に進むまで** の処理を、フロントエンド（`Login.tsx`）→ ルーティング（`urls.py`）→ サーバー処理（`main_views.py`）→ データベース定義（`models.py`）と、**端から端まで本物のコードで追いかける** ことです。1本の処理を最初から最後まで通して見れば、「Webアプリってこう動くのか」が腹の底から分かります。
>
> この章は長いですが、**焦って全部覚える必要はありません。**「こういう地図があったな」と思い出せるように、辞書のように何度も戻ってきてください。

この章で学ぶこと

- **このアプリの目的** — 工数管理とは何か。なぜこのアプリが必要なのか
- **Webアプリの登場人物3者** — ブラウザ・サーバー・データベースの役割分担を、宅配便やレストランにたとえて理解する
- **フロントエンドとバックエンドの分離** — なぜ画面（React）と処理（Django）を別々に作るのか
- **ログイン処理を端から端まで実コードで追う** — `Login.tsx` → `kosu/urls.py` → `main_views.py` → `models.py` を1行ずつ
- **技術スタックと選定理由** — React・TypeScript・Django・PostgreSQL を「7つの基準」で評価する
- **`settings.py` の主要設定を1行ずつ** — アプリの「設定ファイル」の中身を読み解く
- **`hozen_another/urls.py` の交通整理** — どのURLがどこへ振り分けられるのか
- **`index.tsx` の全ルート定義** — どの画面が、どのURLで表示されるのか
- **フォルダ構成の地図** — どこに何があるのか

目次

0. はじめに：この章の読み方
1. このアプリは何をするものか（工数管理とは）
2. Webアプリの3者：ブラウザ・サーバー・データベース
3. フロントエンドとバックエンドの分離
4. 全体像：1回のログインで何が起きるか（鳥瞰図）
5. 実コード①：Login.tsx（フロントの入口）を1行ずつ

6. 実コード②：kosu/urls.py（URLの交通整理）を1行ずつ
7. 実コード③：main_views.py の Login（サーバーの頭脳）を1行ずつ
8. 実コード④：models.py（データの設計図）を1行ずつ
9. 実コード⑤：hozen_another/urls.py（最上位のルーター）を1行ずつ
10. 実コード⑥：index.tsx（フロントの全ルート）を1行ずつ
11. 技術スタックと選定理由（7基準で評価）
12. 実コード⑦：settings.py の主要設定を1行ずつ
13. フォルダ構成の地図
14. トラブルシューティング
15. 演習問題
16. この章のまとめ

---

0. はじめに：この章の読み方

この資料は、**プログラミングが完全に初めての後任者** に向けて書かれています。専門用語が出てきたら、その場で「用語（よみがな：意味）」の形で説明します。たとえ話も多用します。

> **用語: ソースコード（source code）とは？** 人間がテキストエディタで書いた、プログラムの「設計図そのもの」のこと。`.tsx` `.py` `.css` などの拡張子のファイルに書かれた文字列です。これをコンピュータが読み取って実行することで、画面が動きます。本資料では、このソースコードを **実物のまま** 引用して解説します。
>
> **用語: フロントエンド / バックエンド とは？** - **フロントエンド（front-end：フロント、前面）** … ユーザーが直接見て触る「画面」側。本アプリでは **React（リアクト）** で作られています。お店でいう「お客さんが入る売り場」。 - **バックエンド（back-end：バック、背面）** … 画面の裏で、データを保存したり計算したりする「処理」側。本アプリでは **Django（ジャンゴ）** で作られています。お店でいう「裏の倉庫・事務所・厨房」。

この章では、まず概念を説明し、その後で **実際のファイルを開いて、本物のコードを1行ずつ** 読みます。コードの引用箇所には必ず「これは実ファイルの○行目」と分かるように示します。

---

1. このアプリは何をするものか（工数管理とは）

1.1「工数（こうすう）」とは何か

> **用語: 工数（こうすう）とは？**「ある作業に、どれだけの **人の時間** を使ったか」を表す数のこと。たとえば「1人が1時間かけた作業」を基準に、「この作業には3人時（にんじ）かかった」のように数えます。工場や保全（ほぜん：設備の点検・修理）の現場では、「どの作業にどれだけ人手をかけたか」を記録して、ムダを見つけたり、人員計画を立てたりするのに使います。

このアプリは、ひとことで言えば「**従業員が、その日どの作業に何時間使ったかを記録し、集計するシステム**」です。プロジェクトの説明書（`CLAUDE.md`）にもこう書かれています。

> This is a work hours (kosu) management system for tracking employee work time and task categories. （これは、従業員の労働時間と作業区分を記録するための、工数管理システムです。）

1.2 このアプリでできること（画面の種類）

後ほど §10 で実コードを見ますが、このアプリには次のような画面（ページ）があります。先に全体像をつかんでおきましょう。

機能グループ	何をするか	該当フォルダ
ログイン / メニュー	従業員番号で入り、各機能へ案内する	`MainPage/`
工数入力（Kosu）	その日の作業内容・時間を記録、一覧・集計・カレンダー表示	`KosuPage/`
人員（Member）	従業員（社員）の登録・一覧・編集・削除	`MemberPage/`
班員（Team）	チーム（班）の構成と、班員の工数入力状況の確認	`TeamPage/`
工数区分定義（Definition）	「どんな作業区分があるか」の定義を管理	`DefinitionPage/`
問い合わせ（Inquiry）	要望・不具合・問い合わせの送受信	`InquirPage/`
管理者（Administrator）	全体設定・全工数管理・操作履歴の確認	`AdministratorPage/`

> **なぜ「区分定義」が独立しているの？**「作業の種類」は会社や現場によって違い、また時期によって変わります。それを **コードに直接書き込んでしまうと、変更のたびにプログラムを書き換える** ことになり大変です。そこで「作業区分の一覧」を **データとして管理** できるようにしてあります（`kosu_division` モデル。§8で登場）。これは「設定を、プログラムから切り離す」という、よい設計の典型例です。

1.3 このアプリを誰が使うか

- **一般の従業員** … 自分の工数を入力する（権限なし）
- **組長・班長クラス** … 自分に加え、班員の入力状況を確認する（`authority` ＝権限あり）
- **管理者** … 全体の設定変更、全工数の管理、履歴確認ができる（`administrator` ＝管理者フラグあり）

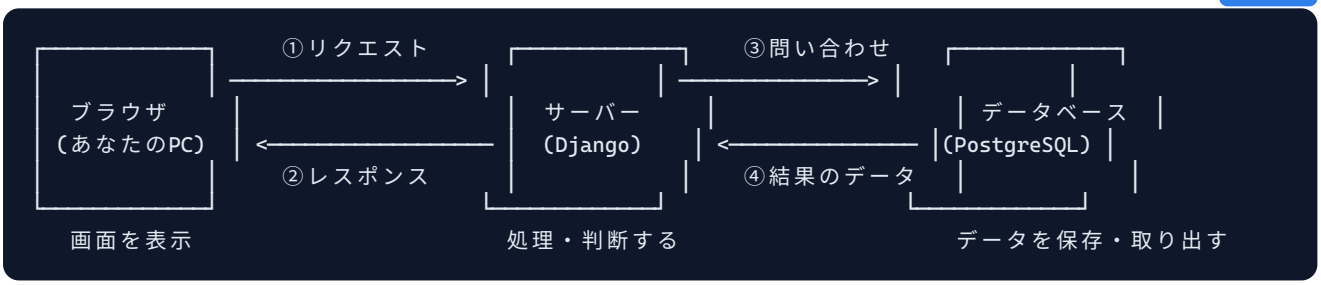
この「3段階の権限」は、後で `member`（人員）モデルの `authority`（権限）と `administrator`（管理者）という2つの真偽値で表現されているのを見ます（§8）。サーバー側の処理（`main_views.py`）でも、`if not member_data.authority:` のように権限チェックが随所に入っています（§7で実例）。

---

2. Webアプリの3者：ブラウザ・サーバー・データベース

Webアプリを理解する最大のコツは、「**登場人物は3人いる**」と知ることです。この3人がメッセージをやり取りして、画面が動きます。

コピー



2.1 3者の役割を「レストラン」でたとえる

登場人物	Webでの役割	レストランでたとえると	本アプリでの実体
ブラウザ	画面を表示し、ユーザーの操作を受け取る	お客さん（席に座って注文する）	Chrome / Edge などで開いた `Login.tsx` の画面
サーバー	注文を受けて処理し、結果を返す	厨房（注文を受けて料理を作る）	Django（`main_views.py` などのPythonコード）
データベース	データを安全に保管・検索する	食材を保管する冷蔵庫・倉庫	PostgreSQL（`models.py` で設計した表）

2.2 やり取りの流れ（4ステップ）

1. **リクエスト（request：要求）** … ブラウザがサーバーに「ログインしたい。従業員番号は12345だ」とお願いします。
2. **サーバーの処理** … サーバーが「その番号は登録されているか？」とデータベースに問い合わせます。
3. **データベースの応答** … データベースが「ある／ない」を返します。

4. **レスポンス（response：応答）** … サーバーが「成功」または「エラー」をブラウザに返し、ブラウザが画面を切り替えます。

> **用語: リクエストとレスポンスとは？** - **リクエスト** … ブラウザ → サーバー へ送る「お願い」。「この情報をくれ」「このデータを保存してくれ」など。 - **レスポンス** … サーバー → ブラウザ へ返す「返事」。お願いの結果（成功・失敗・データ本体）が入っています。この「お願い→返事」が、Webアプリの **すべての通信の基本単位** です。本アプリでも、画面を開くたび・ボタンを押すたびに、このやり取りが何度も発生しています。
>
> **用語: HTTP（エイチティーティーピー）とは？** ブラウザとサーバーが会話するときの「言葉のルール（プロトコル：約束事）」。HyperText Transfer Protocol の略。`http://localhost:8000` の `http` がこれです。リクエストには「**メソッド**」という種類があり、本アプリでは主に次の4つを使います。
>
> **用語: HTTPメソッド（GET / POST / PUT / DELETE）とは？** リクエストの「目的」を表すラベルです。レストランの注文票の種類だと思ってください。 - **GET（ゲット）** … 「データをください」（一覧を見る、画面を開く）。例：メニュー画面を表示。 - **POST（ポスト）** … 「新しいデータを送ります」（登録、ログイン）。例：ログイン情報を送る。 - **PUT（プット）** … 「既存のデータを書き換えます」（更新）。例：工数の修正。 - **DELETE（デリート）** … 「このデータを消してください」（削除）。例：工数の削除。§7で見る `main_views.py` には、`def post(self, request):`（POSTのとき）、`def get(self, request):`（GETのとき）のように、**メソッドごとに処理が分かれて** 書かれています。

2.3 「localhost」と「ポート番号」

開発中、あなたのPCの中でブラウザもサーバーも動かします。そのとき出てくる住所が `localhost`（ローカルホスト：このPC自身）です。

- フロントエンド（React開発サーバー）… `http://localhost:3000`
- バックエンド（Django）… `http://localhost:8000`

> **用語: ポート番号とは？** `:3000` や `:8000` の数字。1台のPCの中で、複数のプログラムが同時に通信できるように付ける「部屋番号」のようなもの。同じPC（localhost）でも、3000番の部屋にはReact、8000番の部屋にはDjangoが住んでいる、というイメージです。なぜ番号で分けるかは、後で `settings.py` の `CORS`（コルス）設定（§12）で関係してきます。

---

3. フロントエンドとバックエンドの分離

3.1 なぜ画面と処理を別々に作るのか

昔ながらのWebアプリは、サーバーが「完成したHTMLページ」を毎回まるごと作ってブラウザに送っていました。本アプリは違い、**フロントエンド（React）とバックエンド（Django）をきっぱり分けて** います。

コピー

```
【昔の方式】
ブラウザ ──「ページちょうだい」──> サーバー ──> HTMLを丸ごと組み立てて返す
（ボタンを押すたびに、画面が真っ白になって、全部読み込み直し）

【本アプリの方式（フロント・バック分離）】
ブラウザ(React) ──「データだけちょうだい（JSON）」──> サーバー(Django)
                <──「はい、データです（JSON）」──────
（Reactが受け取ったデータで、画面の必要な部分だけを書き換える。画面は真っ白にならない）
```

用語: JSON（ジェイソン）とは？ JavaScript Object Notation の略。データを「名前: 値」のペアの集まりで表す、文字列の書き方。人間にも読みやすく、機械も解釈しやすいので、フロントとバックの間でデータをやり取りする「共通語」として使われます。たとえばログイン成功時、サーバーは次のようなJSONを返します（§7で実物を見ます）。

コピー

```
{"status": "success"}
```

「status（状態）という名前の値が success（成功）だ」という意味です。

3.2 分離のメリット（後任者にとっての利点）

メリット	説明
担当を分けやすい	「画面の見た目」は `frontend/` だけ、「処理・データ」は `kosu/` だけ、と探す場所がはっきりする
画面がサクサク	データだけ受け取り、必要な部分だけ書き換えるので、画面全体の再読み込みがない
役割が明快	「これは見た目の問題（フロント）か、データの問題（バック）か」で原因を切り分けやすい
再利用しやすい	同じバックエンドAPIを、別の画面やアプリからも呼べる

保守のヒント: バグが出たとき、まず「これは画面（フロント）の問題か、処理（バック）の問題か」を切り分けると、調査がぐっと速くなります。**画面の見た目・文言・配置** → **`frontend/src/`**、**データの中身・計算・保存** → **`kosu/`** が出発点です。

3.3 「API」という言葉

用語: API（エーピーアイ）とは？ Application Programming Interface の略。難しそうですが、ここでは**「フロントとバックがデータをやり取りするための窓口（URL）」**くらいに考えてください。本アプリでは、`/api/login/` `/api/main_menu/` のように、`/api/` で始まるURLが「バックエンドへの注文窓口」です。フロントは、この窓口にリクエストを送り、JSONで返事を受け取ります。

つまり本アプリの構図は、こうです。

コピー

```
React（frontend/） ──/api/○○/へリクエスト──> Django（kosu/） ──> PostgreSQL
（見た目・操作）              （JSONでやり取り）          （処理・判断）        （保存）
```

この「`/api/` が窓口」という事実は、後で `hozen_another/urls.py` （§9）に実コードで書かれているのを確認します。

---

4. 全体像：1回のログインで何が起きるか（鳥瞰図）

ここからが本章の核心です。「**ログインボタンを押す**」という、たった**1つの操作** を題材に、複数のファイルをまたいで処理を追いかけます。まず全体の流れを地図で見て、その後 §5〜§8 で各ファイルを1行ずつ読みます。

> **▼ このあと追いかける処理の流れ（先に日本語で）**：① ユーザーが従業員番号を入力し「ログイン」ボタンを押す（ `Login.tsx` ） → ② Reactが `/api/login/` へ、番号を入れたPOSTリクエストを送る（ `Login.tsx` ） → ③ Djangoが「 `/api/login/` は誰が担当？」とURL表を引く（ `hozen_another/urls.py` → `kosu/urls.py` ） → ④ 担当の `Login` クラスの `post` 処理が動く（ `main_views.py` ） → ⑤「その番号は人員データにあるか？」をデータベースに問い合わせる（ `models.py` の `member` 表） → ⑥ あれば「成功」、なければ「エラー」のJSONを返す（ `main_views.py` ） → ⑦ Reactが返事を見て、成功ならメインメニューへ画面遷移、失敗ならエラー表示（ `Login.tsx` → `index.tsx` ）

コピー

```
[ブラウザ] Login.tsx
   │  ① 従業員番号 12345 を入力 → ログインボタン押下
   │  ② api.post("/api/login/", { employee_no: 12345 })
   ▼
[サーバー] Django
   │  ③ hozen_another/urls.py が「/api/ → kosu.urls」、kosu/urls.py が「login/ → Login」と振り分け
   │  ④ main_views.py の Login.post() が実行される
   │  ⑤ member.objects.filter(employee_no=12345).exists() で
   ▼     データベースに「この番号いる？」と問い合わせ
[DB] PostgreSQL（models.py で設計した member 表）
   │  ⑥ 「いる」or「いない」を返す
   ▼
[サーバー] Django
   │  ⑦ いれば {"status":"success"} / いなければ {"status":"error", ...}
   ▼
[ブラウザ] Login.tsx
      ⑧ success なら navigate("/")（メインメニューへ）/ error ならエラー表示
      ⑨ index.tsx のルート定義が「/」→ MainMenu を表示
```

この1本の処理を完全に理解できれば、本アプリの **他のすべての機能も同じパターン** で動いていることが分かります。工数入力も、人員登録も、削除も、「フロントがリクエスト → urlsが振り分け → viewsが処理 → modelsの表を読み書き →

JSONで返す」という同じ骨格です。**この章で1本通せば、残りは応用です。**

---

5. 実コード①：Login.tsx（フロントの入口）を1行ずつ

それでは実物のコードを読みます。最初は、ユーザーが最初に触れる画面 `frontend/src/MainPage/Login.tsx`（全71行）です。

> この章では **処理（JavaScript/TypeScript）の流れ** に注目します。画面の見た目（JSXとCSS）の詳しい解説は `03_Web基礎_HTML_CSS.md` にあります。ここでは「ログインの処理が、どこからどこへつながっているか」を中心に読みます。

5.1 ファイル冒頭：import（道具の取り寄せ）— 1〜6行目

> **▼ このコードがやること（先に日本語で）**：このファイルが使う「道具（部品やライブラリ）」を、他のファイルから取り寄せます。料理でいう「材料と調理器具を、棚から作業台に出す」工程です。

コピー

```
// ① React本体と、状態管理の道具useStateを取り寄せ
import React, { useState } from "react";
// ② 画面リンクと、画面移動の道具を取り寄せ
import { Link, useNavigate } from "react-router-dom";
// ③ サーバーへ通信する自前の道具(axios設定済み)
import api from "../api/axios";
// ④ 通信ライブラリ本体(エラー判定に使う)
import axios from "axios";
// ⑤ ロゴ画像ファイルを取り寄せ
import logo from "../img/TitleRogo.png";
// ⑥ この画面専用のCSS(見た目)を取り寄せ
import styles from "../styles/MainPage/Login.module.css";
```

- ① `import` は「他のファイルから道具を持ってくる」命令。`React` は画面を作る本体、`useState`（ユーズステート）は後で説明する「状態を覚えておく道具」。
- ② `useNavigate`（ユーズナビゲート）は「別の画面へ移動する」道具。ログイン成功後にメインメニューへ飛ばすのに使います。
- ③ `api` は、このアプリ専用に設定済みの通信道具です。`../api/axios` というファイルで「サーバーの住所」や「クッキーを送る設定」がまとめてあり、毎回書かずに済みます。
- ④ `axios`（アクシオス）本体。ここでは主に「これは通信エラーかどうか」を判定する `axios.isAxiosError` に使います。

用語: import（インポート）とは？「別ファイルで用意された機能を、このファイルに取り込む」命令。`from "..."` の部分が「どのファイルから」を表します。`"react"` のように名前だけのものは **ライブラリ**（外部から入れた既製品）、`"../api/axios"` のように `./` `../` で始まるものは **このアプリ内の自作ファイル** です（`../` は「1つ上のフォルダ」の意味）。

用語: ライブラリ（library）とは？「よく使う機能を、誰かがまとめて作って公開してくれた部品集」。自分でゼロから作らず、取り寄せて使います。`React`・`axios`・`react-router-dom` はすべてライブラリです。図書館（library）から本を借りるように、機能を借りて使うイメージ。

5.2 コンポーネントの宣言と「状態」— 8〜11行目

▼ このコードがやること（先に日本語で）：ログイン画面という「部品（コンポーネント）」を定義し、その中で「入力された従業員番号」と「エラーメッセージ」を覚えておくための変数（状態）を用意します。

コピー

```
// ① Loginという画面部品を定義(関数コンポーネント)
const Login: React.FC = () => {
  // ② 入力された従業員番号を覚える箱(初期値は空文字)
  const [employee_no, setNumber] = useState<string>("");
  // ③ エラーメッセージを覚える箱(初期値は空文字)
  const [errorMessage, setErrorMessage] = useState<string>("");
  // ④ 画面移動の道具を使えるように準備
  const navigate = useNavigate();
```

- ① `const Login: React.FC = () => { ... }` は「`Login` という名前の **コンポーネント**（画面部品）を作る」という宣言。`React.FC` は「これはReactの関数コンポーネントです」という型の注釈（TypeScriptのしるし）。
- ②③ `useState`（状態フック）が、この章でいちばん大事な道具です。

用語: コンポーネント（component）とは？ 画面を構成する「部品」のこと。ボタン1つ、フォーム1つ、画面1枚を、それぞれ独立した部品として作り、組み合わせて全体を作ります。レゴブロックのイメージ。`Login` は「ログイン画面まるごと1部品」です。

用語: state（ステート：状態）と useState とは？ **state（状態）** とは「画面が覚えておく、変化する値」のこと。ここでは「いま入力欄に何が入っているか（`employee_no`）」「エラーは出ているか（`errorMessage`）」を覚えています。`const [値, 値を変える関数] = useState<型>(初期値);` という形で使います。- `employee_no` … 現在の値（読むときに使う） - `setNumber` … 値を変えるための関数（書き換えるときに使う） - `useState<string>("")` …「文字列（string）型の状態を、初期値は空文字 `""` で作る」 **なぜ普通の変数ではダメ？** Reactでは、stateを `setNumber(...)` で書き換えると、**画面が自動で描き直される** からです。普通の変数を書き換えても、Reactは気づかず画面が更新されません。「stateを変える＝画面を更新する合図」と覚えてください。

5.3 ログイン処理の本体：handleSubmit ― 13〜31行目

ここがこの画面の心臓部です。ボタンを押したときに実行される処理を、丁寧に読みます。

> **▼ このコードがやること（先に日本語で）**：「ログイン」ボタンが押されたら、(1) 画面の標準動作を止め、(2) 古いエラー表示を消し、(3) 入力された番号をサーバーの `/api/login/` へ送り、(4) 返事が「成功」ならメインメニューへ移動、(5) 失敗ならエラーメッセージを表示する ―― という一連の流れを行います。

コピー

```
// ① 送信時に呼ばれる関数(asyncは「待つ処理」を含む印)
const handleSubmit = async (event: React.FormEvent) => {
  // ② ブラウザ標準の「ページ再読み込み」を止める
  event.preventDefault();
  // ③ 前回のエラー表示を消す(空文字にする)
  setErrorMessage("");

  // ④ ここから「失敗するかもしれない処理」
  try {
    // ⑤ サーバーへ番号を送る
    const response = await api.post("/api/login/", { employee_no: Number(employee_no) });
    // ⑥ サーバーからの返事の中身(JSON)を取り出す
    const data = response.data;
    // ⑦ 返事が「成功」だったら
    if (data.status === "success") {
      // ⑧ メインメニュー("/")へ画面移動
      navigate("/");
    // ⑨ 「成功」以外だったら
    } else {
      // ⑩ サーバーのメッセージを表示
      setErrorMessage(data.message || "サーバーエラーが発生しました。");
    }
  // ⑪ 通信そのものが失敗したとき(例外発生)
  } catch (err) {
    // ⑫ それが通信由来のエラーなら
    if (axios.isAxiosError(err)) {
      // ⑬ 401(未認証)なら、ログイン画面へ戻す
      if (err.response?.status === 401) navigate("/login");
      // ⑭ それ以外は、サーバーのメッセージを表示
      else setErrorMessage(err.response?.data.message);
    // ⑮ 通信以外の謎のエラー
    } else setErrorMessage("不明なエラーが発生しました。IT担当者に連絡してください。");
  }
};
```

1行ずつ意味を確認します。

- ① `const handleSubmit = async (event) => { ... }` …「ログインボタンが押されたとき（フォーム送信時）に呼ばれる関数」を定義。`event`（イベント）には「何が起きたか」の情報が入っています。

> **用語: async / await（エイシンク / アウェイト）とは？** サーバーとの通信は「お願いしてから返事が来るまで、少し時間がかかる」処理です。`await`（アウェイト：待つ）は「この処理の返事が来るまで、ここで待ってから次へ進む」という命令。`await` を使う関数には、頭に `async`（非同期：あとで返ってくる）を付ける決まりです。「料理を注文して、出来上がるのを待ってから食べる」イメージ。

- ② `event.preventDefault()` … フォームは標準では「送信するとページが再読み込みされる」動作をします。Reactでは自分で処理したいので、この標準動作を **止めます**。これを忘れると画面が一瞬リロードされてしまいます。
- ③ `setErrorMessage("")` … 前回の失敗で出ていたエラー表示を、いったん消します（§5.2の `errorMessage` state を空にする）。
- ④ `try {` …「この中の処理は失敗するかもしれない。失敗したら `catch`（⑪）へ飛ぶ」という構文。

> **用語: try / catch（トライ / キャッチ）とは？**「失敗するかもしれない処理」を `try { }` で囲み、もし失敗（例外）が起きたら `catch { }` の中で受け止める仕組み。通信は「サーバーが落ちている」「ネットが切れた」など失敗しうるので、必ずこれで守ります。綱渡りの下に張る「安全ネット」のイメージ。

- ⑤ `const response = await api.post("/api/login/", { employee_no: Number(employee_no) });` … **ここが通信の本体** です。
- `api.post(...)` … POSTメソッド（新規送信。§2.2）で、`/api/login/` という窓口へリクエストを送る。
- `{ employee_no: Number(employee_no) }` … 送るデータ。`Number(...)` で **文字列を数値に変換** しています。

> **なぜ `Number(...)` で変換するの？** `<input type="number">` の入力値は、JavaScript側では実は **文字列**（例: `"12345"`）として届きます。サーバー側は数値の従業員番号を期待しているため、`Number("12345")` → `12345` と数値化してから送っています。

- `await` が付いているので、サーバーの返事が来るまでここで待ち、返事を `response` に入れます。
- ⑥ `const data = response.data;` … 返事の中から「中身（JSONデータ）」を取り出して `data` へ。
- ⑦ `if (data.status === "success")` … 中身の `status` が `"success"` か判定。`===` は「完全に等しいか」の比較（型まで一致を確認する厳密比較）。
- ⑧ `navigate("/")` … 成功なら、`"/"`（メインメニュー）へ画面を移動。`useNavigate`（§5.2④）で用意した道具を使っています。
- ⑨⑩ 成功以外なら、サーバーが返したメッセージ（`data.message`）を画面に表示。`||`（オア）は「左がなければ右を使う」。メッセージが無いときは「サーバーエラーが発生しました。」を出します。
- ⑪ `catch (err)` … 通信そのものが失敗したとき（サーバーがエラーを返した、ネットが切れた等）にここへ来ます。
- ⑫ `axios.isAxiosError(err)` … そのエラーが「通信由来のものか」を判定。

- ⑬ `if (err.response?.status === 401) navigate("/login")` … サーバーが **401** を返したら、ログイン画面へ戻します。

  > **用語: ステータスコード（status code）とは？** サーバーが「処理の結果がどうだったか」を3桁の番号で表す共通ルール。代表的なもの： - **200** … 成功（OK） - **400** … リクエストが不正（Bad Request。入力ミスなど） - **401** … 未認証（Unauthorized。ログインが必要） - **403** … 権限なし（Forbidden。ログインはしているが、この操作は許されない） - **404** … 見つからない（Not Found） - **500** … サーバー内部エラー 本アプリでは「401ならログイン画面へ飛ばす」という方針があり、サーバー側（§7）でも未ログイン時に401を返しています。フロントとバックで **401＝再ログイン** という約束が一致しているわけです。 `?.`（オプショナルチェーン）は「`err.response` が存在すれば `.status` を見る、無ければ何もしない（エラーにしない）」という安全な書き方です。

- ⑭⑮ それ以外のエラーは、サーバーのメッセージ、または「不明なエラー」を表示します。

> **この17行が、本アプリの「通信処理の型」です。** 工数入力も人員登録も、「`try` で囲んで `api.post` / `api.get` し、`status` を見て分岐、`catch` でエラー処理」という **同じパターン** を繰り返します。ここを理解すれば、他の画面の処理も読めます。

5.4 画面部分（JSX）の処理に関わる箇所 — 33〜69行目

画面の見た目は `03` 章で詳しく扱うので、ここでは「**処理とどうつながっているか**」だけを拾います。

コピー

```
  // ① ここから画面の中身を返す
  return (
    // ② 画面全体を囲む箱
    <div className={styles["login-wrapper"]}>
      // ③ ロゴ画像
      <img src={logo} alt="業務工数システムロゴ" className={styles["login-logo"]} />
      // ④ 入力フォーム枠
      <form
        // ⑤ 送信時に handleSubmit(§5.3)を呼ぶ ← ここが要!
        onSubmit={handleSubmit}
        // ⑥ キー入力時の処理(Enter誤送信の防止)
        onKeyDown={(e) => {
          if (e.key === "Enter" && e.target instanceof HTMLInputElement && e.target.type !== "textarea") {
            //    Enterでの送信を止め
            e.preventDefault();
            //    入力欄からフォーカスを外す
            (e.target as HTMLInputElement).blur();
          }
        }}
      >
        // ⑦ 見出し
        <h2>ログイン</h2>
        // ⑧ 入力欄をまとめる箱
        <div className={styles["search-bar"]}>
          // ⑨ ラベル
          <label htmlFor="numberInput">従業員番号</label>
          // ⑩ 入力欄
          <input
            //    数値入力
            type="number"
            id="numberInput"
            //    表示する値は state の employee_no(§5.2)
            value={employee_no}
            //    入力が変わるたびに
            onChange={(e) => {
              const value = e.target.value;
              //    state を更新(setNumber。§5.2)← ここが要!
              setNumber(value === "" ? "" : value);
            }}
            //    入力必須
            required
            className={styles["input-focus"]}
          />
          // ⑪ errorMessage(§5.2)があるときだけ表示
          {errorMessage && (
            //    エラーを赤いボックスで表示
            <div role="alert">{errorMessage}</div>
          )}

          // ⑫ 送信ボタン
          <button type="submit" className="blue_button">ログイン</button>
        </div>
      </form>
      // ⑬ ヘルプ画面へのリンク
      <Link to="/help" className={styles["help-button"]}></Link>
    </div>
  );
};

// ⑭ このLogin部品を、他から使えるよう公開する
export default Login;
```

処理の観点で重要なのは次の4点です。

- ⑤ `onSubmit={handleSubmit}` … フォームが送信されると、§5.3で定義した `handleSubmit` が呼ばれます。**「ボタンを押す」と「処理が動く」をつなぐ線** がここです。
- ⑩の `value={employee_no}` と `onChange` … 入力欄の表示は state（`employee_no`）と連動し、文字が変わるたびに `setNumber` で state を更新します。これにより「入力欄に打った値が、`handleSubmit` で送る値と常に一致」します。この「入力欄とstateを結びつける」やり方を **制御コンポーネント** と呼びます（`05` 章で詳述）。
- ⑪ `{errorMessage && (...)}` … `errorMessage` が空でないとき（つまりエラーがあるとき）だけ、赤いエラーボックスを表示。`handleSubmit` の中で `setErrorMessage(...)` した内容がここに出ます。
- ⑭ `export default Login;` … この `Login` 部品を「外部から使えるように公開」します。これにより `index.tsx`（§10）から `import Login` できるのです。

> **用語: export（エクスポート）とは？** `import`（取り寄せ）の逆。「このファイルで作ったものを、他のファイルから使えるように外へ公開する」命令。`export default Login` は「このファイルの代表として `Login` を公開」という意味。§10で `index.tsx` がこれを `import Login from './MainPage/Login';` と受け取ります。

`Login.tsx` のまとめ: ユーザーが番号を入力（state更新）→ ボタンで `handleSubmit` 発動 → `/api/login/` へPOST → 返事で分岐（成功なら画面移動、失敗ならエラー表示）。次は、この `/api/login/` というリクエストが、サーバー側で **どこへ振り分けられるか** を見ます。

---

6. 実コード②：kosu/urls.py（URLの交通整理）を1行ずつ

`Login.tsx` が送った `/api/login/` というリクエストは、Django側で **「どのURLを、どの処理が担当するか」の対応表** に照らして振り分けられます。その対応表が `kosu/urls.py`（全106行）です。

> **用語: ルーティング（routing）/ URLディスパッチとは？** 「来たリクエストのURLを見て、担当の処理へ振り分けること」。電話の交換手が「経理部への電話ですね、おつなぎします」と回すのに似ています。Djangoでは、この対応表を `urls.py` というファイルに書きます。

6.1 ファイル冒頭：import ― 1〜10行目

> **▼ このコードがやること（先に日本語で）**：URL対応表を書くのに必要な道具と、各機能の処理が書かれたファイル群（views）を取り寄せます。

コピー

```
# ① URL定義の道具 path を取り寄せ
from django.urls import path
# ② ログイン等の処理が入ったファイル
from .views import main_views
# ③ 工数入力の処理
from .views import kosu_views
# ④ 人員の処理
from .views import member_views
# ⑤ 班員の処理
from .views import team_views
# ⑥ 工数区分定義の処理
from .views import def_views
# ⑦ 問い合わせの処理
from .views import inquiry_views
# ⑧ バックアップ等の非同期処理
from kosu.views import asynchronous_views
# ⑨ 設定ファイル(settings.py)の値を読む道具
from django.conf import settings
# ⑩ メディアファイル配信の道具
from django.conf.urls.static import static
```

- ① `from django.urls import path` … `path(...)`（パス）は「このURLは、この処理が担当」という1行を書くための道具です。
- ②〜⑧ `main_views`・`kosu_views` などは、機能ごとに分けられた **処理の置き場**（views）。`from .views import ...` の `.`（ドット）は「このアプリ（`kosu`）の中の」という意味です。

> **用語: views（ビュー）とは？** Djangoで「リクエストを受け取って、処理して、レスポンスを返す」役割の場所。本アプリでは機能ごとにファイルを分けています（`main_views.py` ＝ログイン等、`kosu_views.py` ＝工数入力、…）。「厨房の中で、料理ジャンルごとに担当を分けている」イメージ。

6.2 URL対応表の本体：urlpatterns — 14〜103行目

`urlpatterns`（ユーアールエル・パターンズ）という名前のリスト（一覧）に、URLと処理の対応を1行ずつ並べます。代表的な行を読みましょう。

> **▼ このコードがやること（先に日本語で）**：「このURLが来たら、この処理クラスを呼ぶ」という対応を、機能の数だけ並べた一覧です。

コピー

```
# ① URL対応表(リスト)のはじまり
urlpatterns = [
  # ② /login/ → main_views の Login が担当
  path('login/', main_views.Login.as_view(), name='login'),
  # ③ /logout/ → Logout が担当
  path('logout/', main_views.Logout.as_view(), name='logout'),
  # ④ /help/ → Help が担当
  path('help/', main_views.Help.as_view(), name='help'),
  # ⑤ /help_pass/ → HelpPass が担当
  path('help_pass/', main_views.HelpPass.as_view(), name='help_pass'),
  # ⑥ /main_menu/ → Menu が担当
  path('main_menu/', main_views.Menu.as_view(), name='main_menu'),
  # ⑦ 工数メニュー
  path('kosu_menu/', main_views.KosuMenu.as_view(), name='kosu_menu'),
  # ⑧ 工数の新規入力
  path('kosu_new/', kosu_views.KosuNew.as_view(), name='kosu_new'),
  # ... (中略：工数・定義・人員・班員・問い合わせ・管理者の各URLが続く) ...
  # ⑨ ID付きURL(後述)
  path('kosu_update/<int:pk>/', kosu_views.KosuUpdate.as_view(), name='kosu_update'),
```

`path` の中身は3つの部分でできています。`path('login/', main_views.Login.as_view(), name='login')` を分解すると：

1. **`'login/'`** … URLの文字列（パターン）。`/api/login/` の `api/` を除いた部分。
2. **`main_views.Login.as_view()`** … このURLが来たときに呼ぶ処理。「`main_views` ファイルの中の `Login` クラス」を担当に指定。`.as_view()` は「クラスを、URLから呼べる形に変換する」おまじない。
3. **`name='login'`** … この対応に付ける「あだ名」。コードの他の場所からURLを参照するのに使えます。

> **ここで Login.tsx とつながります！** `Login.tsx`（§5.3⑤）が `api.post("/api/login/", ...)` を送ると、Djangoは `/api/` の後ろの `login/` を見て、この `urls.py` の **②の行** にたどり着きます。そして担当の `main_views.Login` を呼び出します。**フロントの1行と、バックの1行が、URLという糸でつながっている** のが見えますか？これがWebアプリの基本構造です。

⑨の `path('kosu_update/<int:pk>/', ...)` の `<int:pk>` は「ここに整数（int）が入る。それを `pk` という名前で処理に渡す」という意味です。たとえば `/api/kosu_update/42/` なら、`pk=42` として処理に届きます。

> **用語: pk（ピーケー：主キー）とは？** Primary Key の略。データベースの各行に自動で付く「重複しない背番号（ID）」。`/kosu_update/42/` は「ID番号42の工数データを編集する」という意味です。§8の `models.py` で、各表に自動でこのID（背番号）が振られる仕組みを見ます。

6.3 同じ処理を共有する非同期URL群 — 79〜97行目

コピー

```
    # 工数のバックアップ
    path('kosu_backup/', asynchronous_views.backup, name='kosu_backup'),
    # 工数の読み込み(復元)
    path('kosu_load/', asynchronous_views.backup, name='kosu_load'),
    # 定義のバックアップ
    path('def_backup/', asynchronous_views.backup, name='def_backup'),
    # ... 同様に member / team / inquiry / setting などのバックアップ・復元が並ぶ ...
```

ここは「いろいろなバックアップ・復元URL」が、**すべて同じ** `asynchronous_views.backup` **という処理を共有** しているのが特徴です。どのデータを扱うかは、URL名やリクエストの中身で見分けます（詳細は別章のバックアップ機能で扱います）。`.as_view()` が付いていないのは、これがクラスではなく **関数** だからです（関数ビューはそのまま渡せます）。

6.4 末尾：メディアファイル配信 — 105行目

コピー

```
# メディアファイル用URLを追加
urlpatterns += static(settings.MEDIA_URL, document_root=settings.MEDIA_ROOT)
```

`+=` は「リストに付け足す」。`static(...)` は「アップロードされたファイル（バックアップファイルなど）を、URLからダウンロードできるようにする」設定です。`settings.MEDIA_URL`・`settings.MEDIA_ROOT` は `settings.py`（§12）で定義された値を読んでいます。

> **`urls.py` のまとめ:** これは **電話交換表** です。`/api/login/` が来たら `Login`、`/api/kosu_new/` が来たら `KosuNew`、…と、URLと担当処理を1対1で対応させています。**「この画面の処理はどこ？」と思ったら、まずこの `urls.py` でURLから担当のviewを探す** のが調査の出発点になります。

---

7. 実コード③：main_views.py の Login（サーバーの頭脳）を1行ずつ

`urls.py` が「`/login/` の担当は `main_views.Login`」と振り分けました。いよいよ、その担当処理 `kosu/views/main_views.py` の中の `Login` クラスを読みます。ここが **サーバーの頭脳** です。

7.1 ファイル冒頭：import — 1〜17行目

> **▼ このコードがやること（先に日本語で）：** サーバー処理に必要な道具（日付・JSON・Djangoの部品・データベースのモデル・データ変換の道具）を取り寄せます。

コピー

```
# ① 日付・時刻を扱う道具
import datetime
# ② OS(ファイルパス等)を扱う道具
import os
# ③ Pythonの実行環境を扱う道具
import sys
# ④ ログ(動作記録)を残す道具
import logging
# ⑤ .env(秘密設定)を読む道具
import environ
# ⑥ JSONを扱う道具
import json
# ⑦ ファイルの場所を扱う道具
from pathlib import Path
# ⑧ HTMLを返す道具
from django.shortcuts import render
# ⑨ JSONで返事を返す道具
from django.http import JsonResponse
# ⑩ ビューの基本クラス
from django.views import View
# ⑪ 設定値を読む道具
from django.conf import settings
# ⑫ API用のビュー基本クラス(よく使う)
from rest_framework.views import APIView
# ⑬ API用の返事を返す道具(よく使う)
from rest_framework.response import Response
# ⑭ ステータスコードの名前付き定数
from rest_framework import status
# ⑮ DBのモデル群を取り寄せ
from ..models import member, Business_Time_graph, kosu_division, team_member, administrator_data, AsyncTask, History
# ⑯ データ変換器
from .serializers import MemberSerializer, AdministratorSerializer, KosuSerializer, DefSerializer, TaskSerializer, HistorySerializer
# ⑰ ページ分割等の補助道具
from ..utils.main_utils import CustomPagination, get_all_model_names_in_myapp
```

特に重要なのは次の3つです。

- ⑫ `APIView` … 「APIのビューはこれを土台に作る」という基本クラス。本アプリのほぼ全ビューがこれを継承しています。
- ⑬ `Response` … フロントへJSONで返事を返す道具。`Login.tsx`（§5.3⑥）が受け取る `response.data` の中身は、これで作られます。
- ⑮ `..models import member, ...` … データベースの表（モデル）を取り寄せ。`..` は「2つ上のフォルダ（`kosu/`）」の意味。`member`（人員表）を使って「この従業員番号は登録されているか」を調べます。

> **用語: モデル（model）とは？** Djangoで「データベースの表（テーブル）」をPythonのクラスとして表したもの。1つのモデル＝1つの表。`member` モデルは「人員表」、`Business_Time_graph` モデルは「工数記録の表」です。§8で `models.py` を読み、これらの設計図を確認します。
>
> **用語: シリアライザ（serializer）とは？** データベースから取り出した「Pythonのオブジェクト」を、フロントへ送れる「JSON」に変換する翻訳器。逆に、フロントから来たJSONをDBに保存できる形に戻す役目もします。`MemberSerializer` は「人員データ ⇄ JSON」の翻訳器です。

7.2 ログイン処理：Login クラス — 78〜102行目

いよいよ核心です。`Login.tsx` がPOSTしたリクエストを、この `Login` クラスの `post` メソッドが受け取ります。

▼ **このコードがやること（先に日本語で）**：フロントから送られてきた従業員番号を取り出し、(1) JSONとしてきちんと読めるか確認し、(2) その番号が人員データ（ `member` 表）に登録されているか調べ、(3) 登録があれば「ログイン状態」を覚え（セッション保存）、使用する工数区分定義の最新版も覚えて「成功」を返し、(4) 登録がなければ「エラー」を返します。

長い行を折り返しているので、なぞってコピーすると行が分かれます。正しくは → コピー

```
# ログイン
# ① ログイン処理(APIView を土台に)
class Login(APIView):
  # POST処理
  # ② POSTリクエストが来たときの処理
  def post(self, request):
    # ③ 失敗しうる処理の開始
    try:
      # 送られてきたデータ確認
      # ④ リクエスト本文をJSONとして読み込む
      data = json.loads(request.body)
      # ⑤ その中から従業員番号を取り出す
      input_number = data.get('employee_no')
    # ⑥ JSONとして読めなかったら
    except json.JSONDecodeError:
      # ⑦ 400エラーを返す
      return JsonResponse({'status': 'error', 'message': 'JSON形式が正しくありません。'},
status=status.HTTP_400_BAD_REQUEST)

    # POSTされた従番が人員データにある場合、セッションに保存&最新工数区分定義取得
    # ⑧ その番号が member 表にあるか?
    if member.objects.filter(employee_no=input_number).exists():
      # ⑨ あれば「ログイン中の番号」を覚える(セッション)
      request.session['login_No'] = input_number
      # ⑩ 工数区分定義の最新版を取得
      def_Ver = kosu_division.objects.order_by("id").last()

      # 取得した工数区分定義をセッションに保存
      # ⑪ もし定義が1件も無ければ
      if not def_Ver:
        # ⑫ エラー
        return JsonResponse({'status': 'error', 'message': '利用可能な工数区分がありません。ERROR052'},
status=status.HTTP_400_BAD_REQUEST)
      # ⑬ 定義があれば
      else:
        # ⑭ その定義名を覚える(セッション)
        request.session['input_def'] = def_Ver.kosu_name
        # ⑮ 「成功」を返す ← Login.tsx の⑦へ!
        return JsonResponse({'status': 'success'})

    # ⑯ 番号が member 表に無ければ
    else:
      # ⑰ エラー
      return JsonResponse({'status': 'error', 'message': '入力された従業員番号は登録がありません。
ERROR048'}, status=status.HTTP_400_BAD_REQUEST)
```

1行ずつ読みます。

- ① `class Login(APIView):` … 「`Login` というAPIのビューを作る」。括弧の中の `APIView` は「これを土台にする（継承する）」という意味。

> **用語: クラス（class）と継承（けいしょう）とは？** **クラス** は「ある種類の処理・データをひとまとめにした設計図」。**継承** は「既存のクラスの機能を引き継いで、自分のクラスを作る」こと。`Login(APIView)` は「`APIView` が持つ便利機能（リクエストの受け取り方など）を引き継いだ上で、ログイン専用の処理を足す」という意味です。料理の「基本のレシピを土台に、自分流のアレンジを足す」イメージ。

- ② `def post(self, request):` … 「POSTリクエストが来たら、この処理を実行」。`request`（リクエスト）に、フロントから送られた情報すべてが入っています。`def` は「関数（処理のまとまり）を定義する」Pythonの命令です。

> **なぜ `post` という名前？** §2.2のHTTPメソッドと対応しています。`Login.tsx` が `api.post(...)`（POST）で送ったので、サーバー側も `def post` が反応します。もしGETで来たら `def get` が呼ばれます。**メソッド名が、HTTPメソッドと一致** しているのがDRF（後述）のお約束です。

- ③〜⑦ `try / except` … `json.loads(request.body)` でリクエスト本文をJSONとして読み込みます。`data.get('employee_no')` で、その中から従業員番号を取り出します。もし送られた中身がJSONとして壊れていたら（`JSONDecodeError`）、400エラー（不正なリクエスト）を返します。

> **⑤ の `.get('employee_no')` に注目:** これは `Login.tsx`（§5.3⑤）が `{ employee_no: Number(employee_no) }` という名前で送ったデータと **キー名が一致** しています。フロントが `employee_no` という名前で送り、バックが `employee_no` という名前で受け取る —— この **名前の一致** が、フロントとバックの「契約」です。どちらかが名前を変えると通信が壊れます。

- ⑧ `if member.objects.filter(employee_no=input_number).exists():` … **データベースへの問い合わせ** です。ここが §4 の鳥瞰図で「⑤データベースに問い合わせ」と書いた部分。
- `member.objects` … `member` 表（人員表。§8で設計を見ます）の全データ。
- `.filter(employee_no=input_number)` … 「従業員番号が、入力された番号と一致する行だけ」に絞り込み。
- `.exists()` … 「そういう行が1つでもあるか？（True / False）」を返す。

> つまりこの1行は「**入力された従業員番号は、人員表に登録されているか？**」をデータベースに尋ねています。これがログイン認証の本体です。本アプリのログインはパスワードではなく「番号が登録されているか」で判定する方式です。

- ⑨ `request.session['login_No'] = input_number` … 登録があれば、**セッション** に「ログイン中の従業員番号」を記録します。

> **用語: セッション（session）とは？**「この人はログイン済みだ」という状態を、サーバー側が一定期間覚えておく仕組み。ログインのたびにブラウザに小さな合言葉（クッキー）を渡し、次回以降のリクエストでその合言葉を見て「あ、さっきログインした人だ」と分かるようにします。これにより、画面を移動しても **毎回番号を入れ直さずに済む** のです。§7.4で、他のメニュー画面がこのセッションを使って「ログイン済みか」を確認しているのを見ます。

- ⑩ `def_Ver = kosu_division.objects.order_by("id").last()` … 工数区分定義（作業区分の一覧。§8の `kosu_division` ）の中から、**ID順で最後＝最新版** を取り出します。`.order_by("id")` で番号順に並べ、`.last()` で最後の1件を取得。
- ⑪⑫ もし定義が1件も無ければ（ `not def_Ver` ）、`ERROR052` という番号付きエラーを返します。

> **なぜエラーに番号（ERROR052）を付けるの？** ユーザーが「エラーが出た」と言ってきたとき、番号があれば「どこで起きたエラーか」をコードから即座に特定できます。保守をラクにする工夫です。

- ⑭ `request.session['input_def'] = def_Ver.kosu_name` … 最新定義の名前もセッションに記録。以後の工数入力で「どの定義を使うか」の基準になります。
- ⑮ `return JsonResponse({'status': 'success'})` … **すべてOKなら「成功」を返します。**この `{'status': 'success'}` というJSONが、ネットワークを通って `Login.tsx` （§5.3⑥⑦）の `data.status === "success"` で受け取られ、`navigate("/")` でメインメニューへ進みます。**ここで処理が1周してフロントへ戻る**のです。
- ⑯⑰ 番号が登録されていなければ、`ERROR048` 「入力された従業員番号は登録がありません」を400エラーで返します。これは `Login.tsx` （§5.3⑭）の `catch` 側で受け取られ、エラーメッセージとして画面に表示されます。

> **これで §4 の鳥瞰図が完成しました。** `Login.tsx` （番号入力・送信）→ `urls.py` （ `/login/` を `Login` へ振り分け）→ `main_views.py` （ `member` 表に問い合わせ・成功/失敗を判定）→ JSONで返事 → `Login.tsx` （成功なら画面遷移）。**1本の処理が、複数のファイルをまたいで完結する** ―― この骨格が、本アプリのすべての機能に共通します。

7.3 もう1つの例：Help クラスの初期化処理 ― 106〜171行目

`Login` だけでなく、`main_views.py` には多くのクラスがあります。代表として `Help` （ヘルプ画面の初期化処理）を軽く見ておきます。これは「データを初期状態（工場出荷状態）に戻す」処理です。

> **▼ このコードがやること（先に日本語で）**：ヘルプ画面から「人員データを初期化」「定義を初期化」「設定を初期化」のいずれかが要求されたら、その表を全削除して、最低限のデフォルトデータ（管理者1名・定義1つ・設定1つ）を作り直します。

コピー

```
class Help(APIView):
  def post(self, request):
    # ① 送られてきたデータ
    post_data = request.data
    # ② 人員初期化の要否
    memberReset = post_data.get('memberReset')
    # ③ 定義初期化の要否
    defReset = post_data.get('defReset')
    # ④ 設定初期化の要否
    settingReset = post_data.get('settingReset')

    try:
      # ⑤ 人員初期化が要求されたら
      if memberReset:
        # ⑥ 人員表を全削除
        member.objects.all().delete()
        # ⑦ デフォルト管理者(番号12345)を作成
        default_member = member(employee_no=12345,
                                name='admin',
                                shop='その他',
                                #    権限あり
                                authority=True,
                                #    管理者
                                administrator=True,
                                #    各種休憩時間の初期値(後略)
                                break_time1='#11401240',
                                # ...(休憩時間の設定が続く) ...
                                )
        # ⑧ データベースに保存
        default_member.save()
      # ... (defReset / settingReset も同様にデフォルトを作り直す) ...
      # ⑨ 成功
      return Response({'status': 'success', 'message': 'OK'}, status=status.HTTP_200_OK)
    # ⑩ 何か失敗したら
    except Exception as e:
      return Response({'status': 'error', 'message': 'データの初期化中にエラーが発生しました。'}, status=status.HTTP_400_BAD_REQUEST)
```

注目点：

- ⑦ **のデフォルト管理者** … `employee_no=12345`、`authority=True`（権限）、`administrator=True`（管理者）で作られます。**もし全データを初期化したら、この `12345` でログインすればよい** ということです。これは引き継ぎで非常に重要な知識です。
- ⑥ `.all().delete()` …「全部消す」。⚠️ 危険な操作です。本番データに対して安易に実行してはいけません。

> ⚠️ **重要な注意:** `Help` クラスの初期化処理は **全データを消す** 強力な機能です。後任者として、これがどういう機能か（緊急時にデフォルト状態へ戻すためのもの）を理解しておき、**本番環境で誤って実行しない** よう気をつけてください。

7.4 認証チェックの共通パターン — Menu クラス 204〜237行目

`main_views.py` の各メニュー画面（`Menu`・`MemberMenu` など）には、**ほぼ同じ「認証チェック」のパターン** が繰り返し現れます。`Menu`（メインメニュー）を例に見ます。

> **▼ このコードがやること（先に日本語で）**：メインメニュー画面を開くとき、(1) セッションに「ログイン中の番号」があるか確認し、(2) 無ければ401エラー（ログインへ戻れ）を返し、(3) あればその人員データと設定データを取り出してフロントへ返します。

コピー

```
class Menu(APIView):
  # ① GET(画面表示)のとき
  def get(self, request):
    # ② セッションからログイン番号を取り出す
    login_no = request.session.get('login_No')
    # ③ セッションから定義名を取り出す
    def_ver = request.session.get('input_def')

    # ④ ログイン番号が無ければ(未ログイン)
    if not login_no:
      # ⑤ 401
      return Response({'status': 'error', 'message': 'ログイン情報が確認できません。'}, status=status.HTTP_401_UNAUTHORIZED)
    # ⑥ 定義名も無ければ
    if not def_ver:
      return Response({'status': 'error', 'message': '使用する工数区分定義情報が確認できません。'}, status=status.HTTP_401_UNAUTHORIZED)

    try:
      # ⑦ その番号の人員データを取得
      member_data = member.objects.get(employee_no=login_no)
    # ⑧ もし見つからなければ
    except member.DoesNotExist:
      return Response({'status': 'error', 'message': 'ユーザー情報が見つかりません。'}, status=status.HTTP_401_UNAUTHORIZED)
    # ... 設定データ取得、シリアライザでJSON化して返す ...
```

ここで `Login.tsx`（§5.3⑬）の「401ならログイン画面へ」が効いてきます。`Menu` 画面を開いたとき、**もしログインしていなければ ⑤ で401を返し**、フロントはそれを見て自動的にログイン画面へ戻します。「**未ログインの人を、ログイン画面へ追い返す**」**仕組みが、フロント（401を見て遷移）とバック（401を返す）の協力で実現** されているのです。

> **保守のヒント:** `main_views.py` の各メニュークラスは、この「セッション確認 → 無ければ401 → あれば人員データ取得 → 権限チェック → データをJSONで返す」という **判で押したような構造** をしています。1つ理解すれば、残りはほぼ同じパターンの繰り返しです。`MemberMenu`（258行付近）では、加えて `if not member_data.authority:`（権限が無ければ403）という **権限チェック** が入っているのが違いです。

---

8. 実コード④：models.py（データの設計図）を1行ずつ

§7で `member.objects.filter(...)` のように、データベースの表に問い合わせていました。その **表の設計図** が `kosu/models.py` です。「どんなデータを、どんな形で保存するか」を定義しています。

> **用語: データベース / テーブル / レコード / カラム とは？**（表計算ソフトでたとえる） - **データベース（database：DB）** … データを保管する「倉庫」全体。Excelブック1冊のイメージ。 - **テーブル（table：表）** … 倉庫の中の1つの「表（シート）」。本アプリの `member`（人員）・`Business_Time_graph`（工数）などが、それぞれ1枚の表。 - **レコード（record：行）** … 表の1行。「従業員1人ぶんの情報」「工数記録1件」など。 - **カラム（column：列）** … 表の1列。「従業員番号」「氏名」など、データの項目。Djangoでは、**1つのモデル（クラス）＝1つのテーブル、モデルの中の1つの変数＝1つのカラム** に対応します。

8.1 ファイル冒頭 — 1〜2行目

コピー

```
# ① Djangoのモデル機能を取り寄せ
from django.db import models
# ② JSON型のカラムを使う道具(Historyで使用)
import jsonfield
```

8.2 member モデル（人員表） — 6〜65行目

ログイン認証で使われた、最重要の表です。

> **▼ このコードがやること（先に日本語で）**：従業員1人ぶんの情報（従業員番号・氏名・所属ショップ・権限・管理者フラグ・各勤務形態ごとの休憩時間など）を保存する「人員表」の設計図です。

コピー

```
# ① member表の設計図(models.Model を継承)
class member(models.Model):
  # ② 所属ショップの選択肢一覧
  shop_list = [
    #    (左：保存される値, 右：画面表示)
    ('P', 'P'),
    ('R', 'R'),
    ('W1', 'W1'),
    # ... (W2, T1, T2, A1, A2, J など) ...
    ('その他', 'その他'),
    ('組長以上(P,R,T,その他)', '組長以上(P,R,T,その他)'),
    ('組長以上(W,A)', '組長以上(W,A)'),
    #    異動・退社した人の区分
    ('異動・退社', '異動・退社'),
  ]

  # ③ 従業員番号(整数)← ログイン認証で使う鍵!
  employee_no = models.IntegerField('従業員番号')
  # ④ 氏名(最大100文字)
  name = models.CharField('氏名', max_length=100)
  # ⑤ 所属(②の選択肢から選ぶ)
  shop = models.CharField('ショップ', choices=shop_list, max_length=15)
  # ⑥ 権限あり/なし(真偽値)← 班員機能などの可否
  authority = models.BooleanField('権限')
  # ⑦ 管理者か(真偽値)← 管理者機能の可否
  administrator = models.BooleanField('管理者')
  # ⑧ 1直(勤務形態)の昼休憩時間
  break_time1 = models.CharField('1直昼休憩時間', max_length=9)
  # ⑨ 1直の残業休憩1
  break_time1_over1 = models.CharField('1直残業休憩時間1', max_length=9)
  # ... (break_time1〜6、各 over1〜3 まで、勤務形態×休憩区分の組み合わせが続く) ...
  # 休憩エラー判定の有効/無効
  break_check = models.BooleanField('休憩エラー有効チェック', null=True)

  # ⑩ この行を文字で表すときの表現
  def __str__(self):
    #    氏名を返す(管理画面などで見やすく)
    return self.name
```

1行ずつ、カラム定義の読み方を学びます。

- ③ `employee_no = models.IntegerField('従業員番号')` … 「`employee_no` という名前の、整数（Integer）型の列。表示名は『従業員番号』」。**これが §7.2⑧ の `filter(employee_no=...)` で照合された鍵** です。

  > **用語: フィールド型（Field）とは？** カラムの「データの種類」を指定するもの。`IntegerField` ＝整数、`CharField` ＝短い文字列、`TextField` ＝長い文字列、`BooleanField` ＝True/False、`DateField` ＝日付、`DateTimeField` ＝日時。型を決めることで、データベースが「この列には数字しか入らない」などと管理してくれます。

- ④ `name = models.CharField('氏名', max_length=100)` … 文字列の列。`max_length=100` は「最大100文字」。`CharField` は必ず最大長を指定します。
- ⑤ `shop = models.CharField('ショップ', choices=shop_list, max_length=15)` … `choices=shop_list`（②の一覧）を指定すると、**決められた選択肢からしか選べない列** になります。画面ではドロップダウンになります。
- ⑥⑦ `authority`・`administrator` … `BooleanField`（真偽値＝True/False）。§1.3で説明した「3段階の権限」を、この2つの旗（フラグ）で表現しています。§7.4で `if not member_data.authority:` のように使われていました。
- ⑧⑨ `break_time1` ～ … 勤務形態（1直・2直・3直・常昼・連1直・連2直の6種）ごとに、昼休憩＋残業休憩3つ、計24個の休憩時間カラムが並びます。`max_length=9` で、`#11401240` のような独自形式（`#` ＋ 開始時刻4桁 ＋ 終了時刻4桁）で休憩時間を表します。

  > **なぜ休憩時間がこんなに多いの？** 工場では勤務シフト（直）ごとに休憩時刻が違うためです。工数を正しく計算するには「いつ休憩で、いつ働いていたか」が必要なので、人ごと・勤務形態ごとに休憩時刻を持たせています。

- ⑩ `def __str__(self): return self.name` … この行（レコード）を文字で表すときの見た目。Django管理画面などで「ID番号」ではなく「氏名」で表示され、人間に分かりやすくなります。

> **用語: null=True / blank=True とは？** モデル定義でよく見る指定。`null=True` は「データベース上で空（未設定）を許す」、`blank=True` は「入力フォーム上で空欄を許す」。`pop_up1 = models.CharField(..., null=True, blank=True)` のように、必須でない項目に付けます。
>
> **`pk`（背番号）はどこ？** member モデルには `id` の定義が見当たりませんが、Djangoは **自動で `id` という主キー（§6.2の pk）を各モデルに付けます**。これが §6.2 の `/kosu_update/<int:pk>/` で使われる背番号です。`settings.py`（§12）の `DEFAULT_AUTO_FIELD` がこの自動IDの型を決めています。

8.3 Business_Time_graph モデル（工数記録の表）— 69～87行目

工数入力の本体データです。「誰が・いつ・何の作業を・何時間したか」を1件ずつ記録します。

コピー

```
# ① 工数記録の表
class Business_Time_graph(models.Model):
  # ② 記録した人の従業員番号
  employee_no3 = models.IntegerField('従業員番号')
  # ③ 人員表への参照
  name = models.ForeignKey(member, verbose_name='氏名', null=True, on_delete=models.SET_NULL)
  # ④ 使った定義のVer
  def_ver2 = models.CharField('工数区分定義Ver', max_length=100, blank=True, null=True)
  # ⑤ 就業日(日付)
  work_day2 = models.DateField('就業日')
  # ⑥ 勤務シフト(直)
  tyoku2 = models.CharField('直', max_length=2, blank=True, null=True)
  # ⑦ 作業内容(コード化)
  time_work = models.CharField('作業内容', max_length=288, blank=True, null=True)
  # ⑧ 作業の詳細
  detail_work = models.CharField('作業詳細', max_length=32676, blank=True, null=True)
  # ⑨ 残業時間
  over_time = models.IntegerField('残業時間', blank=True, null=True)
  # ⑩ 昼休憩時間
  breaktime = models.CharField('昼休憩時間', max_length=9, blank=True, null=True)
  # ...(残業休憩1〜3、就業形態など)...
  # ⑪ 入力が完了/正常か(OK/NG)
  judgement = models.BooleanField('工数入力OK_NG', null=True)
  # ⑫ 休憩を変更したか
  break_change = models.BooleanField('休憩変更チェック', null=True)

  def __str__(self):
    return str(self.id) + '__' + str(self.work_day2) + ':' + str(self.employee_no3)
```

- ③ `name = models.ForeignKey(member, ...)` … **外部キー（ForeignKey）** です。

> **用語: 外部キー（Foreign Key：FK）とは？**「別の表のレコードを指し示す」カラム。ここでは工数記録から `member`（人員表）の特定の人を指しています。「この工数は、人員表の○○さんのもの」という **表どうしのつながり** を作ります。Excelで「別シートの行を参照する」イメージ。 `on_delete=models.SET_NULL` は「もし参照先の人員が削除されたら、このカラムを空（NULL）にする」という意味。人員を消しても工数記録自体は消えず、紐付けだけ外れます。

- ⑪ `judgement`（工数入力OK_NG）… この工数記録が「ちゃんと入力完了したか」の旗。班員フォロー機能（`TeamMenu`）では、この `judgement` が `False` なら「工数未入力があります」と通知しています。

8.4 kosu_division モデル（工数区分定義）— 115〜319行目

§7.2⑩で「最新版を取得」していた、作業区分の定義表です。このモデルは **非常に長い** ので、構造だけ理解します。

コピー

```
# ① 工数区分定義の表
class kosu_division(models.Model):
  # ② 定義のバージョン名
  kosu_name = models.CharField('工数区分定義Ver名', blank=True, null=True, max_length=100)
  # ③ 区分1の名前
  kosu_title_1 = models.CharField('工数区分名1', blank=True, null=True, max_length=100)
  # ④ 区分1の定義
  kosu_division_1_1 = models.TextField('定義1', blank=True, null=True)
  # ⑤ 区分1の作業内容
  kosu_division_2_1 = models.TextField('作業内容1', blank=True, null=True)
  # ⑥ 区分1が変動か固定か
  kosu_division_3_1 = models.BooleanField('変動/固定1', null=True)
  # ... kosu_title_2 ～ kosu_title_50、それぞれ _1～_3 が続く（最大50区分） ...
  kosu_title_50 = models.CharField('工数区分名50', blank=True, null=True, max_length=100)
  kosu_division_1_50 = models.TextField('定義50', blank=True, null=True)
  kosu_division_2_50 = models.TextField('作業内容50', blank=True, null=True)
  kosu_division_3_50 = models.BooleanField('変動/固定50', null=True)

  def __str__(self):
    return str(self.id) + ' : ' + str(self.kosu_name)
```

ポイントは「**1つの定義レコードに、作業区分を最大50個ぶん（`_1`～`_50`）持たせている**」ことです。各区分は「区分名（`kosu_title_n`）・定義（`kosu_division_1_n`）・作業内容（`kosu_division_2_n`）・変動/固定（`kosu_division_3_n`）」の4つで構成されます。

> **なぜ50個も列を並べるの？（設計の評価）** 本来、データベースの教科書的には「区分」を別の表に分けて、1レコード1区分にするのが綺麗です。しかし本アプリは「1つの定義Ver＝1レコードに50区分を横並び」という方式を採っています。これは扱いがシンプルになる反面、列数が膨大になります。**後任者として「なぜこうなっているか」を理解し、安易に作り直そうとしない** ことが大切です（動いているものには理由があります）。§7.2⑩ の `kosu_division.objects.order_by("id").last()` が、この表の「最新の1レコード（＝最新の定義一式）」を取り出していたわけです。

8.5 その他のモデル（一覧）

残りのモデルも、役割だけ押さえておきましょう。

モデル	役割	特徴的なカラム
`team_member`（91行〜）	班（チーム）の構成。最大15人を登録	`member1` 〜 `member15`、`follow`（フォロー機能ON/OFF）
`def_choice`（323行〜）	作業詳細の選択肢（記号と内容のペア）	`def_symbol`（A〜Zなどの記号）、`def_select`
`administrator_data`（334行〜）	全体設定（一覧表示件数・問い合わせ担当者など）	`menu_row`（一覧表示項目数）、`administrator_employee_no1` 〜3
`inquiry_data`（355行〜）	問い合わせ（要望・不具合・問い合わせ）	`content_choice`、`inquiry`（本文）、`answer`（回答）
`AsyncTask`（375行〜）	非同期タスク（バックアップ等）の記録	`task_id`、`status`、`MAX_RECORDS=1000`（古い記録は自動削除）
`Operation_history`（405行〜）	操作履歴	`post_page`、`operation_models`、`status`
`History`（419行〜）	データ変更の監査ログ	`operation`、`changes`（JSON型）、`MAX_RECORDS=500000`

`AsyncTask` と `History` には `save()` メソッドが上書きされており、「レコードが上限（`MAX_RECORDS`）を超えたら、古いものから自動削除する」仕組みが入っています。これにより、ログが無限に溜まってデータベースが膨張するのを防いでいます。

コピー

```
class AsyncTask(models.Model):
  # 保持する最大件数
  MAX_RECORDS = 1000

  # タスクの一意なID
  task_id = models.CharField(max_length=255, unique=True)
  # 状態(pending/success/error)
  status = models.CharField(max_length=50, choices=[
    ('pending', 'Pending'),
    ('success', 'Success'),
    ('error', 'Error')
  ])
  # 結果
  result = models.TextField(null=True, blank=True)
  # 作成日時(自動)
  created_at = models.DateTimeField(auto_now_add=True)
  # 更新日時(自動)
  updated_at = models.DateTimeField(auto_now=True)

  # 保存時に上限を超えたら古い順に削除
  def save(self, *args, **kwargs):
    super().save(*args, **kwargs)
    current_count = self.__class__.objects.count()
    if current_count > self.MAX_RECORDS:
      excess_count = current_count - self.MAX_RECORDS
      oldest_records = self.__class__.objects.order_by('created_at')[:excess_count]
      for record in oldest_records:
        record.delete()
```

> **`models.py` のまとめ:** ここは **データベースの設計図** です。`member`（人員）でログイン認証し、`Business_Time_graph`（工数）に記録を貯め、`kosu_division`（定義）で作業区分を管理します。**「このデータ項目はどこに保存されている？」と思ったら、まずこの `models.py` で該当モデルのカラムを探す** のが調査の起点になります。

---

9. 実コード⑤：hozen_another/urls.py（最上位のルーター）を1行ずつ

§6で見た `kosu/urls.py` は「`/api/login/` のうち `login/` の部分」を担当していました。では「`/api/` という入口」自体は、どこで決まっているのでしょうか。それが、プロジェクト全体の **最上位のURL対応表** `hozen_another/urls.py`（全18行）です。

用語: プロジェクト と アプリ の違い（Django） Djangoでは、**プロジェクト（project）** が「サイト全体」、**アプリ（app）** が「その中の機能のまとまり」です。本アプリでは `hozen_another/` がプロジェクト（設定・最上位URL）、`kosu/` がアプリ（工数管理の実体）。`hozen_another/urls.py` が「玄関の交通整理」、`kosu/urls.py` が「各部屋の中の案内図」という階層関係です。

▼ このコードがやること（先に日本語で）：ブラウザから来たURLを、その先頭部分で大きく振り分けます。`/admin/` はDjango管理画面へ、`/api/` で始まるものは `kosu` アプリのURL表（§6）へ、それ以外は全部Reactの画面へ、という3つの大きな分岐です。

コピー

```
# ① Django標準の管理画面
from django.contrib import admin
# ② URL定義の道具(include=他のurls.pyを取り込む)
from django.urls import path, include, re_path
# ③ React表示処理とmanifest処理を取り寄せ
from kosu.views.main_views import react_view, manifest
# ④ 設定値を読む
from django.conf import settings
# ⑤ 静的ファイル配信の道具
from django.conf.urls.static import static



# ⑥ 最上位のURL対応表
urlpatterns = [
  # ⑦ /admin/ → Django管理画面
  path('admin/', admin.site.urls),
  # ⑧ /BA で始まるURL → kosu/urls.py へ
  path('BA', include('kosu.urls')),
  # ⑨ /api/ で始まるURL → kosu/urls.py へ ← 重要!
  path('api/', include('kosu.urls')),
  # ⑩ /manifest.json → アプリ情報を返す
  path('manifest.json', manifest),
  # ⑪ admin以外の全URL → Reactの画面を返す
  re_path(r'^(?!admin).*$', react_view),
]

# ⑫ 開発モードのときだけ
if settings.DEBUG:
    # ⑬ 静的ファイル配信を追加
    urlpatterns += static(settings.STATIC_URL, document_root=settings.STATIC_ROOT)
```

1行ずつ読みます。

- ② `include`（インクルード）… 「別の `urls.py` を、ここに取り込む」道具。これで階層的なURL構成ができます。

- ⑦ `path('admin/', admin.site.urls)` … `/admin/` で始まるURLは、Django標準の **管理画面** へ。これはDjangoが用意してくれる、データベースを直接見たり編集したりできる便利な裏画面です。
- ⑧⑨ `path('api/', include('kosu.urls'))` … **ここが §6 とのつながり** です。`/api/` で始まるURLが来たら、後ろの部分を `kosu/urls.py`（§6）に渡して、そこで担当を決めてもらいます。

  > つまり `/api/login/` は、**まずここ（⑨）で「`api/` だから `kosu.urls` へ」と振り分けられ、次に `kosu/urls.py`（§6.2②）で「`login/` だから `Login` へ」と振り分けられる**、2段階のルーティングを通っていたのです。`/BA` でも同じ `kosu.urls` を使えるようにしてあります（別経路でのアクセス用）。

- ⑩ `path('manifest.json', manifest)` … `/manifest.json` はアプリ情報（アプリ名・アイコン色など）を返す処理へ。スマホでホーム画面に追加したときの表示などに使われます。
- ⑪ `re_path(r'^(?!admin).*$', react_view)` … **これが「分離型アプリ」の肝** です。
- `re_path` … 正規表現（パターンマッチ）でURLを指定する道具。
- `r'^(?!admin).*$'` …「`admin` で **始まらない** あらゆるURL」というパターン。
- `react_view` … そのURLに対して、Reactのページ（`index.html`）を返す処理。

  > **なぜこれが必要？** 本アプリはフロントとバックが分離しているので、`/`・`/kosu-menu`・`/member-list` など **画面用のURLは、すべてReactに任せます**。ユーザーがどの画面URLを開いても（adminとapi以外なら）、サーバーは同じReactの土台（`index.html`）を返し、**実際にどの画面を出すかはReact側（§10の `index.tsx`）が決める** のです。これを **SPA（エスピーエー：シングルページアプリケーション）** と呼びます。**用語: SPA（Single Page Application）とは？**「1枚のHTML土台を最初に読み込み、その後は画面の中身だけをJavaScriptで差し替える」方式のWebアプリ。ページ全体を読み込み直さないので、画面遷移が速くスムーズです。本アプリはこの方式です。

- ⑫⑬ `if settings.DEBUG:` … `settings.py` の `DEBUG` が `True`（開発モード）のときだけ、静的ファイル（CSS・画像など）の配信設定を追加します。本番では別の方法（WhiteNoise。§12）で配信するため、ここは開発用です。

> **`hozen_another/urls.py` のまとめ:** これは **玄関の交通整理** です。`/admin/` →管理画面、`/api/` →バックエンド処理（kosu/urls.py 経由）、それ以外→Reactの画面、という3分岐。**フロント（画面URL）とバック（apiURL）の分離が、この1ファイルで実現** されています。

---

10. 実コード⑥：index.tsx（フロントの全ルート）を1行ずつ

§9の⑪で「画面用URLはReactに任せる」と分かりました。では、Reactは「どのURLで、どの画面部品を出すか」を **どこで決めている** のか。それが、フロントエンドの司令塔 `frontend/src/index.tsx`（全180行）です。

用語: ルーター（router）とは？ フロント側で「URLに応じて表示する画面部品を切り替える」道具。本アプリは `react-router-dom`（リアクト・ルーター・ドム）というライブラリを使います。§9⑪でサーバーが「画面はReactに任せた」と言った、その判断を実際に下すのがこのルーターです。

10.1 import：全画面部品の取り寄せ — 1〜53行目

▼ このコードがやること（先に日本語で）：Reactとルーターの道具、共通CSS、そしてアプリの全画面部品（ログイン・各種メニュー・一覧・編集など約50個）を取り寄せます。

コピー

```
// ① React本体
import React from 'react';
// ② Reactを画面に描く道具
import ReactDOM from 'react-dom/client';
// ③ ルーターの道具一式
import { BrowserRouter as Router, Routes, Route, useLocation } from "react-router-dom";
// ④ 全画面共通のCSS(03章で解説)
import './styles/global.css';
// ⑤ §5で読んだログイン画面を取り寄せ
import Login from './MainPage/Login';
// ⑥ ヘルプ画面
import Help from './MainPage/Help';
// ⑦ メインメニュー画面
import MainMenu from './MainPage/MainMenu';
// ⑧ 工数メニュー
import KosuMenu from './KosuPage/KosuMenu';
// ⑨ 工数入力
import KosuNew from './KosuPage/KosuNew';
// ... (KosuList, KosuEdit, ... 工数・定義・人員・班員・問い合わせ・管理者の全画面が続く) ...
// 最後：操作履歴詳細
import AdministratorHistoryDetail from './AdministratorPage/AdministratorHistoryDetail';
```

- ③ ルーターの4つの道具を取り寄せます。
- `BrowserRouter`（`as Router` で `Router` と改名）… ルーター全体を有効にする外枠。
- `Routes`・`Route` …「このURLなら、この部品」という対応を書くための道具。
- `useLocation`（ユーズロケーション）…「今どのURLにいるか」を知る道具。
- ⑤ `import Login from './MainPage/Login';` … §5.4⑭で `export default Login` した、**あのログイン画面** をここで取り寄せています。`export` と `import` が、ファイルをまたいでつながっているのが分かります。
- **⑤以降** 約50個の画面部品をすべて取り寄せます。フォルダ名（`MainPage`・`KosuPage`・`MemberPage` …）が、§1.2の機能グループと対応しているのに注目してください。

10.2 App コンポーネント：タイトル設定とルート定義 — 57〜170行目

コピー

```
// ① アプリ全体を表す部品
const App: React.FC = () => {
  // ② 今いるURLの情報を取得
  const location = useLocation();

  // ③ URLが変わるたびに実行する処理
  React.useEffect(() => {
    // ④ URLごとのページタイトル一覧
    const routeTitles: { [key: string]: string } = {
      //    /login のときのタブ表示名
      "/login": "ログイン - 業務工数システム",
      "/help": "ヘルプ - 業務工数システム",
      "/": "Main Menu - 業務工数システム",
      "/kosu-menu": "工数Menu - 業務工数システム",
      // ...（全URLに対応する日本語タイトルが続く）...
      "/manager-history-detail/:id": "データ操作履歴詳細 - 業務工数システム",
    };

    // ⑤ ブラウザのタブ名を設定
    document.title = routeTitles[location.pathname] || "業務工数システム";
  // ⑥ location(URL)が変わったら③を再実行
  }, [location]);
```

- ③ `React.useEffect(() => {...}, [location])` …「`location`（URL）が変わるたびに、中の処理を実行する」仕組み。

  > **用語: useEffect（ユーズエフェクト）とは？**「画面が表示された後や、特定の値が変わった後に、何か処理をする」ためのReactの道具。`[location]` の部分が「これが変わったら実行」という条件。ここでは「URLが変わったら、ブラウザのタブのタイトルを変える」のに使っています。`05` 章で詳述。

- ④⑤ URLごとに、ブラウザのタブに出すタイトル（例：「ログイン - 業務工数システム」）を決めています。`document.title = ...` でタブ名を変更。`||` は「一覧に無いURLなら『業務工数システム』を表示」という保険。

続いて、本体である **ルート定義** です。

> **▼ このコードがやること（先に日本語で）**：「このURLパスのときは、この画面部品を表示する」という対応を、全画面ぶん並べます。§9⑪でサーバーから渡されたReactが、ここを見て実際の画面を決めます。

コピー

```
  return (
    // ① ルート対応のまとまり
    <Routes>
      // ② /login → Login画面(§5)を表示
      <Route path="/login" element={<Login />} />
      // ③ /help → Help画面
      <Route path="/help" element={<Help />} />
      // ④ / → メインメニュー(ログイン成功後の行き先!)
      <Route path="/" element={<MainMenu />} />
      // ⑤ /kosu-menu → 工数メニュー
      <Route path="/kosu-menu" element={<KosuMenu />} />
      // ⑥ /kosu-new → 工数入力
      <Route path="/kosu-new" element={<KosuNew />} />
      // ⑦ /kosu-list → 工数一覧
      <Route path="/kosu-list" element={<KosuList />} />
      // ⑧ /kosu-update/42 など → 工数編集(:idは可変)
      <Route path="/kosu-update/:id" element={<KosuEdit />} />
      // ...(定義・人員・班員・問い合わせ・管理者の全ルートが続く)...
      // 最後
      <Route path="/manager-history-detail/:id" element={<AdministratorHistoryDetail />} />
    </Routes>
  );
};
```

- ② `<Route path="/login" element={<Login />} />` …「URLが `/login` のとき、`Login` 画面部品（§5）を表示する」。`element={<Login />}` が「表示する部品」です。
- ④ `<Route path="/" element={<MainMenu />} />` … **ここが §5.3⑧ とつながります。** `Login.tsx` が成功時に `navigate("/")` で `/` へ移動すると、ルーターがこの行を見て `MainMenu`（メインメニュー）画面を表示します。**ログイン成功後の行き先が、ここで定義されている** のです。
- ⑧ `path="/kosu-update/:id"` の `:id` …「ここは可変（どんな値でも入る）」を表し、`/kosu-update/42` のようなURLにマッチします。§6.2⑨でサーバー側の `<int:pk>` を見ましたが、**フロント側のルートにも対応する `:id` がある** わけです。

10.3 アプリの起動：root.render — 172〜180行目

> **▼ このコードがやること（先に日本語で）**：作った `App` 部品を、`index.html` の中の `root` という場所に実際に描き込みます。これがアプリ起動の最後の一歩です。

コピー

```
// ① index.html内の#rootを取得
const root = ReactDOM.createRoot(document.getElementById('root') as HTMLElement);

// ② そこにReactアプリを描く
root.render(
  // ③ 開発時の問題検出を有効にする外枠
  <React.StrictMode>
    // ④ ルーターを有効化(URL切替を可能に)
    <Router>
      // ⑤ §10.2で作った App を描画
      <App />
    </Router>
  </React.StrictMode>
);
```

- ① `document.getElementById('root')` … `index.html` の中にある `<div id="root"></div>` を探します。**ここがReactアプリの「描画される場所（土台）」**です。§9⑪でサーバーが返した `index.html` の中の、この `root` に、Reactが画面を流し込みます。
- ④ `<Router>` … これで初めてルーティング（URLに応じた画面切替）が有効になります。`<App />` をこれで包むことで、§10.2のルート定義が効くようになります。
- ③ `<React.StrictMode>` … 開発中に潜在的な問題を見つけやすくする「厳格モード」。本番動作には影響しません。

> **`index.tsx` のまとめ:** これは **フロントの司令塔** です。全画面部品を取り寄せ、URLと画面の対応表（`Routes` / `Route`）を定義し、最後に `index.html` の `root` にアプリを描き込みます。**「この画面URLは、どの部品が表示する？」と思ったら、まずこの `index.tsx` の `<Route>` 一覧を見る** のが起点になります。
>
> **ここまでで、ログインの全行程が完全につながりました：** `Login.tsx`（入力→送信）→ `hozen_another/urls.py`（`/api/` →kosu）→ `kosu/urls.py`（`login/` →Login）→ `main_views.py`（DB照合→成功）→ JSON返却 → `Login.tsx`（`navigate("/")`）→ `index.tsx`（`/` → `MainMenu` 表示）。**6つのファイルが、1本のログイン処理でつながっている** のを実コードで確認しました。

---

11. 技術スタックと選定理由（7基準で評価）

> **用語: 技術スタック（tech stack）とは？** そのアプリを作るのに使っている「技術の組み合わせ」のこと。料理でいう「使っている調理法と食材の一式」。本アプリのスタックは `CLAUDE.md` にまとめられています。

11.1 本アプリの技術スタック一覧

層	技術	役割
フロント言語	TypeScript	型のある安全なJavaScript。画面の処理を書く
フロント本体	React 19	画面を部品（コンポーネント）で組み立てる
UI部品	Material-UI 7	ボタン・表などの既製デザイン部品
ルーティング	React Router DOM 7	URLと画面の対応（§10）
通信	Axios	サーバーとのHTTP通信（§5の `api`）
グラフ・暦	Chart.js / FullCalendar	工数のグラフ表示・カレンダー表示
テスト	Vitest	フロントの自動テスト
バック本体	Django 4.2	サーバー処理の枠組み（§7）
API	Django REST Framework	API（`APIView`・`Response`。§7）を作る道具
非同期処理	Django-Q	バックアップ等の重い処理を裏で実行
データベース	PostgreSQL	データの保存（§8）
静的配信	WhiteNoise	CSS・画像の配信（§12）

11.2 主要技術を「7つの基準」で評価する

各技術が「なぜ選ばれたか」を、後任者が納得できるよう **7つの基準** で評価します。

> **評価の7基準:** ① 学習しやすさ ② 情報の豊富さ（困ったとき調べやすいか） ③ 開発の速さ ④ 保守のしやすさ ⑤ 性能 ⑥ コミュニティ・将来性 ⑦ 本アプリとの相性

React（フロント本体）

基準	評価	理由
①学習	○	部品（コンポーネント）の考え方が直感的。ただしstate等の概念は慣れが必要
②情報	◎	世界中で使われ、日本語情報も豊富
③開発速度	◎	部品を組み合わせるので、似た画面を量産しやすい（本アプリの多数の一覧画面）
④保守	○	部品単位で直せる。stateの流れを追う訓練は要る
⑤性能	◎	必要な部分だけ再描画するため画面が速い（§3のSPA）
⑥将来性	◎	業界標準級。求人も多く後任を探しやすい
⑦相性	◎	多画面・多フォームの本アプリに最適

TypeScript（フロント言語）

基準	評価	理由
①学習	○	JavaScriptに「型」を足したもの。型の分だけ覚えることは増える
②情報	◎	普及済みで情報豊富
③開発速度	○	型注釈を書く手間はあるが、エディタの補完が強力で総じて速い
④保守	◎	**型のおかげで、間違った使い方を書いた瞬間にエラーで気づける。**保守の安全網
⑤性能	—	最終的にJavaScriptに変換されるので実行性能はJSと同等
⑥将来性	◎	フロント開発の事実上の標準
⑦相性	◎	データ項目が多い本アプリで、型による取り違え防止が効く

> **なぜ TypeScript？** §5.2で見た `useState<string>("")` の `<string>` のように、「この値は文字列」と型で宣言します。すると、うっかり数値を入れようとした瞬間にエディタが赤線で警告してくれます。**間違いを実行前に発見できる** ので、初心者の保守でも事故が減ります。

Django（バック本体）

基準	評価	理由
①学習	○	「電池付属（必要機能が最初から揃う）」で、決まった書き方に従えば動く
②情報	◎	歴史が長く、日本語の解説・書籍が多い
③開発速度	◎	管理画面（§9⑦）・認証・ORM（§8）が標準装備で、ゼロから作らずに済む
④保守	◎	`views`・`models`・`urls` と役割でファイルが分かれ、どこを直すか明快（本章で体感した通り）
⑤性能	○	一般的な業務アプリには十分。極端な高負荷は別途工夫
⑥将来性	◎	世界的に広く使われ安定
⑦相性	◎	データ管理・権限・管理画面が必要な業務システムに最適

Django REST Framework（API）

基準	評価	理由
①学習	○	`APIView`・`Serializer` の型が分かれば量産できる（§7で体感）
④保守	◎	リクエスト処理・JSON変換の定型化で、似たAPIを同じ形で書ける
⑦相性	◎	フロント分離型（§3）でJSON APIを作る本アプリの中核

PostgreSQL（データベース）

基準	評価	理由
①学習	○	基本的なSQLが使えればよい。Django経由（§8のORM）なら直接SQLを書く場面は少ない
④保守	◎	堅牢で、データが壊れにくい。トランザクション（途中で失敗したら巻き戻す仕組み）が強い
⑤性能	◎	大量データ・複雑な検索に強い（§7のAdministratorKosuListのような絞り込み検索）
⑥将来性	◎	オープンソースの代表的DB。長期運用に安心
⑦相性	◎	工数記録が蓄積する本アプリのデータ量・検索要件に合う

> **用語: ORM（オーアールエム）とは？** Object-Relational Mapping の略。「データベースの操作を、Pythonのコードで書けるようにする翻訳層」。§7.2⑧の `member.objects.filter(employee_no=...)` のように、SQL（データベース専用言語）を直接書かず、Pythonらしい書き方でDBを操作できます。Djangoの大きな魅力で、§8のモデルがこのORMの土台です。

11.3 総評：このスタックの「性格」

本アプリのスタックは、「**広く使われ・情報が多く・保守しやすい」定番の組み合わせ** です。尖った最新技術より、**長期運用と引き継ぎのしやすさ** を重視した堅実な選択といえます。後任者にとって嬉しいのは、困ったときに **日本語の情報がたくさん見つかる** ことです。

---

12. 実コード⑦：settings.py の主要設定を1行ずつ

最後に、アプリ全体の「設定ファイル」`hozen_another/settings.py`（全202行）の主要部分を読みます。ここは **アプリの動作を決めるダイヤル盤** です。値を間違えると起動しなくなることもあるので、慎重に扱います。

12.1 基本設定と秘密情報 — 1〜23行目

> **▼ このコードがやること（先に日本語で）**：アプリの基準フォルダを決め、秘密情報（鍵）を外部ファイル（.env）から読み込み、開発モードかどうか、どのホスト名からのアクセスを許すかを設定します。

コピー

```
# ① ファイルパスを扱う道具
from pathlib import Path
# ② OS機能(環境変数等)
import os
# ③ .envファイルを読む道具
import environ
# ④ ログを回転(古いものを消す)させる道具
from logging.handlers import RotatingFileHandler



# ⑤ プロジェクトの基準フォルダ(このファイルの2つ上)
BASE_DIR = Path(__file__).resolve().parent.parent

# ⑥ 環境変数読み取り器を準備
env = environ.Env()
# ⑦ .env ファイルを読み込む
env.read_env(os.path.join(BASE_DIR, '.env'))

# ⑧ Djangoの秘密鍵(.envから読む)
SECRET_KEY=env('SECRET_KEY')

# ⑨ 開発モードか(.envから真偽値で読む)
DEBUG=env.bool('DEBUG')

# ⑩ 文字コードはUTF-8(日本語が化けない)
DEFAULT_CHARSET = 'utf-8'

# ⑪ アクセスを許可するホスト名一覧
ALLOWED_HOSTS = [
    # 本番(Azure)のアドレス
    'hozen-kosu-react-cwaqashkafgbg5e5.japaneast-01.azurewebsites.net',
    #    開発用
    'localhost',
    #    開発用(自分自身のIP)
    '127.0.0.1'
    ]
```

- ⑤ `BASE_DIR` … プロジェクトの基準となるフォルダ。以後、ファイルの場所を `os.path.join(BASE_DIR, ...)` のように、ここを起点に組み立てます。

- ⑦⑧ `.env`（ドットエンブ）という **別ファイルから秘密情報を読み込む** のが重要なポイントです。

  > **用語: .env と環境変数 とは？** `.env` は「秘密情報（鍵・パスワードなど）を書いておく専用ファイル」。これを **ソースコードに直接書かず**、別ファイルに分けて管理します。なぜなら、ソースコードはGit（バージョン管理）で共有されることがあり、そこにパスワードを書くと **漏洩** するからです。`.env` はGitに含めない（`.gitignore` で除外する）のが鉄則。`env('SECRET_KEY')` で、その中の値を読み込みます。

- ⑧ `SECRET_KEY` … Djangoが暗号処理（セッションの署名など）に使う秘密の鍵。**絶対に外部に漏らしてはいけません。**
- ⑨ `DEBUG` … `True` なら開発モード（詳しいエラー画面が出る）、`False` なら本番モード（エラー詳細を隠す）。

  > ⚠️ **本番で `DEBUG=True` は厳禁:** 開発モードでは、エラー時に内部情報（コードやファイルパス）が画面に表示されます。本番でこれが見えると攻撃の手がかりになり危険です。本番の `.env` では必ず `DEBUG=False` にします。

- ⑪ `ALLOWED_HOSTS` …「このホスト名（住所）宛のアクセスだけ受け付ける」一覧。本番のAzureアドレスと、開発用の `localhost`・`127.0.0.1` が登録されています。ここに無いホスト名でアクセスすると拒否されます（なりすまし対策）。

12.2 インストール済みアプリ — 25〜38行目

コピー

```
# ① 有効にする機能(アプリ)の一覧
INSTALLED_APPS = [
    #    管理画面(§9⑦)
    'django.contrib.admin',
    #    認証機能
    'django.contrib.auth',
    #    モデル管理の土台
    'django.contrib.contenttypes',
    #    セッション機能(§7.2⑨で使用)
    'django.contrib.sessions',
    #    メッセージ表示
    'django.contrib.messages',
    #    静的ファイル管理
    'django.contrib.staticfiles',
    #    デザイン部品(管理系の一部)
    'bootstrap4',
    #    日付選択UI
    'bootstrap_datepicker_plus',
    #    本アプリ本体(§6〜§8で読んだkosu)!
    'kosu',
    #    非同期処理(Django-Q)
    'django_q',
    #    REST Framework(§7のAPIView等)
    'rest_framework',
    #    CORS(別オリジン通信)対応
    'corsheaders',
]
```

- `'kosu'` … **本章で読んできた `kosu/` アプリが、ここで有効化** されています。これが無いと、`models.py` の表も `views` の処理も認識されません。
- `'rest_framework'` … §7で使った `APIView`・`Response` を提供します。
- `'corsheaders'` … 次の §12.3 のCORS対応。フロント（3000番）とバック（8000番）が **別ポート＝別オリジン** で通信できるようにします。

12.3 ミドルウェアとCORS ― 40〜59行目

コピー

```
# ① リクエストが通る「関所」の一覧(上から順に通過)
MIDDLEWARE = [
    #    CORS処理(最初に置くのが定石)
    'corsheaders.middleware.CorsMiddleware',
    #    セキュリティ
    'django.middleware.security.SecurityMiddleware',
    # セッション(§7のログイン状態保持)
    'django.contrib.sessions.middleware.SessionMiddleware',
    'django.middleware.common.CommonMiddleware',
    #    CSRF(なりすまし投稿)対策
    'django.middleware.csrf.CsrfViewMiddleware',
    'django.contrib.auth.middleware.AuthenticationMiddleware',
    'django.contrib.messages.middleware.MessageMiddleware',
    'django.middleware.clickjacking.XFrameOptionsMiddleware',
    #    静的ファイル配信(WhiteNoise)
    'whitenoise.middleware.WhiteNoiseMiddleware',
    # 自作:現在のリクエスト保持
    'kosu.middleware.clear_session_middleware.CurrentRequestMiddleware',
]

# ② すべてのオリジンからの通信を許可
CORS_ORIGIN_ALLOW_ALL = True
# ③ クッキー(セッション)付き通信を許可
CORS_ALLOW_CREDENTIALS = True
# ④ 明示的に許可するオリジン
CORS_ALLOW_ORIGINS = [
    #    React開発サーバー
    'http://localhost:3000',
    #    Django
    'http://localhost:8000',
    'http://127.0.0.1:8000',
]
```

- ① `MIDDLEWARE`（ミドルウェア）… リクエストがビューに届くまでに通る「関所」の列。

> **用語: ミドルウェア（middleware）とは？** リクエストとレスポンスの間に挟まって、共通処理（セキュリティチェック・セッション復元など）を行う仕組み。空港の「保安検査の列」のイメージ。リクエストは上から順に各関所を通ってビューへ届き、レスポンスは逆順で戻ります。

- ②③④ `CORS`（コルス）の設定。

> **用語: CORS（コルス）とは？** Cross-Origin Resource Sharing。ブラウザには「別オリジン（別ドメイン・別ポート）への通信を、勝手にさせない」安全装置があります。本アプリはフロント（3000番）とバック（8000番）が **別ポート＝別オリジン** なので、このままだと通信がブロックされます。それを「この組み合わせは許可」と明示的に解除するのがCORS設定です。`CORS_ALLOW_CREDENTIALS = True` は「セッションのクッキーも一緒に送ってよい」という、ログイン状態を保つために必須の設定です。

12.4 REST Framework・セッション ― 61〜80行目

コピー

```
# ① REST Frameworkの設定
REST_FRAMEWORK = {
    # ページ分割方式
    'DEFAULT_PAGINATION_CLASS': 'rest_framework.pagination.PageNumberPagination',
    #   1ページ20件(一覧画面の標準)
    'PAGE_SIZE': 20,
    'DEFAULT_PARSER_CLASSES': (
        #   受信データはJSONとして解釈
        'rest_framework.parsers.JSONParser',
    ),
    'DEFAULT_RENDERER_CLASSES': (
        #   返事はJSONで返す
        'rest_framework.renderers.JSONRenderer',
    ),
}

# ② セッションをデータベースに保存
SESSION_ENGINE = 'django.contrib.sessions.backends.db'
# ③ セッションの有効期間(秒)=約10年
SESSION_COOKIE_AGE = 315360000
# ④ クッキーの送信範囲(なりすまし対策)
SESSION_COOKIE_SAMESITE = 'Lax'
# ⑤ HTTPSのみ送信するか(開発はFalse)
SESSION_COOKIE_SECURE = False
# ⑥ 最上位URL表は hozen_another/urls.py(§9)
ROOT_URLCONF = 'hozen_another.urls'
```

- ① `PAGE_SIZE: 20` … 一覧を20件ずつに分けて表示（ページネーション）。§7.4の `AdministratorKosuList` などで使われていた `CustomPagination` の土台設定です。`JSONParser`・`JSONRenderer` により「受信も送信もJSON」と決まり、§3のJSONやり取りが実現します。
- ② `SESSION_ENGINE = ...backends.db` … セッション情報（§7.2⑨で保存した `login_No` 等）を **データベースに保存** する設定。

- ③ `SESSION_COOKIE_AGE = 315360000` … 約10年。事実上「ログインしっぱなし」にする設定です（現場で何度もログインし直す手間を省くため）。
- ⑥ `ROOT_URLCONF = 'hozen_another.urls'` … **「最初に見るURL表は §9 のファイル」と指定**。すべてのリクエストは、まずここから振り分けが始まります。

12.5 テンプレートと静的ファイル — 101〜106行目, 157〜173行目

コピー

```
STATICFILES_DIRS = [
    # ① Reactのビルド成果物(JS/CSS)の場所
    os.path.join(BASE_DIR, 'frontend/build/static'),
]
# Reactのbuild内のindex.htmlをテンプレートとして指定
# ② index.htmlの場所(§9⑪が返すもの)
TEMPLATES[0]['DIRS'] = [os.path.join(BASE_DIR, 'frontend/build')]
```

コピー

```
# ③ 静的ファイルのURL接頭辞
STATIC_URL = '/static/'
# ④ 集約先フォルダ(本番配信用)
STATIC_ROOT = os.path.join(BASE_DIR, 'staticfiles')
# ⑤ WhiteNoiseで圧縮配信
STATICFILES_STORAGE = 'whitenoise.storage.CompressedManifestStaticFilesStorage'
# ⑥ アップロードファイルのURL(§6.4で使用)
MEDIA_URL = '/media/'
# ⑦ アップロードファイルの保存先
MEDIA_ROOT = os.path.join(BASE_DIR, 'media')
```

- ①② ここが **フロントとバックの結合点** です。Reactを `npm run build`（`01` 章で扱う）すると `frontend/build/` に成果物ができ、Djangoはそこの `index.html`（§9⑪が返すもの）と `static/`（JS・CSS）を配信します。**「Reactで作った画面を、Djangoが配る」連携が、この設定で実現** されています。
- ⑤ **WhiteNoise** … §11.1の「静的配信」。本番でCSS・JS・画像を効率よく（圧縮して）配る仕組みです。

12.6 データベース・非同期・ログ — 114〜202行目

コピー

```
# ① データベース接続設定
DATABASES = {
    'default': {
        #    PostgreSQLを使う
        'ENGINE': 'django.db.backends.postgresql',
        #    DB名(.envから)
        'NAME': os.getenv('DB_NAME', ''),
        #    ユーザー名(.envから)
        'USER': os.getenv('DB_USER', ''),
        #    パスワード(.envから)
        'PASSWORD': os.getenv('DB_PASSWORD', ''),
        #    DBの住所(.envから)
        'HOST': os.getenv('DB_HOST', ''),
        #    PostgreSQLの標準ポート
        'PORT': '5432',
    }
}

# Django Q の設定
# ② Django-Q(非同期処理)の設定
Q_CLUSTER = {
    #    タスク管理にDBを使う
    'orm': 'default',
    #    同時に4つの作業者で処理
    'workers': 4,
    'recycle': 2000,
    'retry': 2000,
    #    1タスク最大1800秒(30分)
    'timeout': 1800,
}
```

- ① データベース接続情報。**接続情報（DB名・ユーザー・パスワード・住所）はすべて `.env` から読む**（§12.1で説明した秘密情報の分離）。`postgresql` を使い、標準ポート `5432` で接続します。§8で設計したモデルは、ここで指定したPostgreSQLに保存されます。
- ② `Q_CLUSTER` … バックアップなど時間のかかる処理を **裏で（ユーザーを待たせず）実行** するDjango-Qの設定。`workers: 4` で4つ同時処理、1タスク最大30分。

コピー

```
# ③ 言語は日本語
LANGUAGE_CODE = 'ja'
# ④ タイムゾーンは東京
TIME_ZONE = 'Asia/Tokyo'
# ⑤ タイムゾーンを意識して時刻を扱う
USE_TZ = True
# ⑥ 自動IDの型(§8.2の pk の型)
DEFAULT_AUTO_FIELD = 'django.db.models.BigAutoField'
```

- ③④ 日本語・東京時間の設定。エラーメッセージや日付表示が日本向けになります。
- ⑥ `DEFAULT_AUTO_FIELD` … §8.2で触れた「各モデルに自動で付く `id`（pk）」の型を、大きな整数（`BigAutoField`）に指定。データが大量に増えてもID番号が枯渇しません。

最後の `LOGGING`（175〜202行目）は、サーバーの動作記録を `web_console.log` というファイルに残す設定です。`RotatingFileHandler` で「ログが1MBを超えたら古いものを捨てる」回転方式（`maxBytes: 1024 * 1024 * 1`）にし、ログが無限に膨らむのを防いでいます。

コピー

```
LOGGING = {
    'version': 1,
    'disable_existing_loggers': False,
    # ログの書式
    'formatters': { 'simple': { 'format': '{message}', 'style': '{', }, },
    'handlers': {
        # web_console.log への出力設定
        'web_console': {
            'level': 'INFO',
            'class': 'logging.handlers.RotatingFileHandler',
            'formatter': 'simple',
            'filename': 'web_console.log',
            # 最大ファイルサイズ（1MB）
            'maxBytes': 1024 * 1024 * 1,
            'backupCount': 1,
            'encoding': 'utf-8',
        },
    },
    'loggers': {
        'views_logger': { 'handlers': ['web_console'], 'level': 'INFO', 'propagate': False, },
    },
}
```

> **`settings.py` のまとめ:** これは **アプリの設定ダイヤル盤** です。秘密情報は `.env` から読み（漏洩防止）、`INSTALLED_APPS` で `kosu` を有効化し、`CORS` でフロント・バックの別ポート通信を許し、`DATABASES` でPostgreSQLにつなぎ、`STATICFILES_DIRS` / `TEMPLATES` でReactのビルド成果物を配信します。**本章で見た全ファイルが、この設定によって連携して動いている** のです。

13. フォルダ構成の地図

最後に、プロジェクト全体の「どこに何があるか」の地図を示します。迷ったらここに戻ってください。

コピー

```
hozen-kosu-React/                    ← プロジェクト全体のフォルダ
│
├── manage.py                        ← Djangoの操作コマンド入口(runserver等。01章で使う)
├── .env                             ← 秘密情報(鍵・DBパスワード)。Gitに含めない(§12.1)
├── web_console.log                  ← サーバーの動作ログ(§7.1, §12.6)
├── CLAUDE.md                        ← プロジェクト説明書
│
├── hozen_another/                   ← 【プロジェクト設定】Django全体の設定(§9, §12)
│   ├── settings.py                  ←   設定ダイヤル盤(§12)
│   ├── urls.py                      ←   最上位URL表=玄関の交通整理(§9)
│   └── wsgi.py / asgi.py            ←   本番起動の入口
│
├── kosu/                            ← 【アプリ本体】工数管理の実体(§6〜§8)
│   ├── models.py                    ←   データの設計図(§8)
│   ├── urls.py                      ←   アプリ内URL表(§6)
│   ├── views/                       ←   処理(リクエスト→レスポンス)
│   │   ├── main_views.py            ←     ログイン・各メニュー(§7)
│   │   ├── kosu_views.py            ←     工数入力の処理
│   │   ├── member_views.py          ←     人員の処理
│   │   ├── team_views.py            ←     班員の処理
│   │   ├── def_views.py             ←     工数区分定義の処理
│   │   ├── inquiry_views.py         ←     問い合わせの処理
│   │   ├── asynchronous_views.py    ←     バックアップ等の非同期処理(§6.3)
│   │   └── serializers.py           ←     データ⇄JSON変換器(§7.1)
│   ├── tasks.py                     ←   Django-Qの非同期タスク本体
│   ├── signals.py                   ←   モデル変更の自動追跡(Historyへの記録)
│   ├── middleware/                  ←   関所処理(§12.3)
│   └── utils/                       ←   補助道具(ページ分割等。§7.1)
│
├── staticfiles/                     ← 本番配信用に集約された静的ファイル(§12.5)
├── media/                           ← アップロードファイル(バックアップ等。§12.5)
│
└── frontend/                        ← 【フロントエンド】React側(§5, §10)
    ├── package.json                 ←   フロントの設定・依存ライブラリ一覧(01章)
    ├── build/                       ←   ビルド成果物(Djangoが配信。§12.5)
    │   ├── index.html               ←     Reactの土台(§9⑪が返す)
    │   └── static/                  ←     ビルド済みJS/CSS
    └── src/                         ←   フロントのソースコード(ここを編集する)
        ├── index.tsx                ←     司令塔=全ルート定義(§10)
        ├── api/axios                ←     通信設定(§5.1③の api)
        ├── styles/                  ←     CSS(見た目。03章)
        │   ├── global.css           ←       全画面共通
        │   └── 各ページ別/.module.css  ←       画面専用
        ├── Components/              ←     共通部品(Pagination, Loading等)
        ├── MainPage/                ←     ログイン・メニュー(Login.tsx等。§5)
        ├── KosuPage/                ←     工数入力の各画面
        ├── MemberPage/              ←     人員の各画面
        ├── TeamPage/                ←     班員の各画面
        ├── DefinitionPage/          ←     工数区分定義の各画面
        ├── InquirPage/              ←     問い合わせの各画面
```

```
├── AdministratorPage/          ←   管理者の各画面
└── tests/                      ←   フロントの自動テスト(Vitest)
```

> **地図の使い方（保守の出発点）:** - **画面の見た目・文言を直したい** → `frontend/src/` の該当ページフォルダの `.tsx`、見た目なら `styles/` の `.css` - **画面の処理（送信・計算）を直したい** → 同じく該当ページの `.tsx` - **データの保存内容・計算・権限を直したい** → `kosu/views/` の該当 `_views.py` - **保存するデータ項目を変えたい** → `kosu/models.py`（変更時はマイグレーションが必要。`01` 章） - **どのURLがどこへ行くか調べたい** → `kosu/urls.py`（§6）と `hozen_another/urls.py`（§9） - **全体設定を変えたい** → `hozen_another/settings.py`（§12）

---

14. トラブルシューティング

この章の内容に関連して、引き継ぎ初期に出会いやすいトラブルと対処をまとめます。

症状	考えられる原因	対処
ログインしても「従業員番号は登録がありません」	その番号が `member` 表に無い（ERROR048）	Django管理画面（`/admin/`）で人員を確認・登録。初期化後なら番号 `12345`（§7.3）
ログイン後すぐログイン画面に戻される	セッションが保持されていない（§7.4で401）	`settings.py` の `CORS_ALLOW_CREDENTIALS=True`（§12.3）と、フロントの通信がクッキー送信する設定か確認
フロントからAPIを呼ぶと通信エラー（CORS）	別オリジンがブロックされた	`settings.py` の `CORS_ALLOW_ORIGINS`（§12.3）にフロントのアドレスがあるか確認
「利用可能な工数区分がありません」（ERROR052）	`kosu_division` 表が空（§7.2⑪）	ヘルプ画面の定義初期化（§7.3）か、Django管理画面で定義を1件登録
画面は出るがデータが出ない	バックエンド（Django）が起動していない	`python manage.py runserver` でDjangoを起動（`01` 章）
画面そのものが真っ白	フロント（React）が起動していない／ビルド未実施	開発時は `npm start`。本番配信なら `npm run build`（§12.5, `01` 章）
起動時に SECRET_KEY のエラー	`.env` が無い／鍵が未設定（§12.1）	`.env` ファイルを用意し `SECRET_KEY` 等を設定
データベース接続エラー	`.env` のDB情報が誤り／DB未起動（§12.6）	`DB_NAME`・`DB_USER`・`DB_PASSWORD`・`DB_HOST` を確認し、PostgreSQLが起動しているか確認
URLを直接開くと404やエラー	ルーティングの理解不足	`/api/...` はバック（§6, §9⑨）、画面URLはReact（§9⑪, §10）。両者を混同していないか確認

15. 演習問題

手を動かして、この章の地図を体に入れましょう。**まずは「読んで探す」演習** から始め、コードの変更は慎重に（壊しても安全なローカル環境で）行ってください。

1. **処理の追跡（ペンと紙で）**：ログインボタンを押してからメインメニューが出るまで、本章で読んだ6ファイルを **どの順で通るか** を、矢印で書き出してみる（§4・§10の最後のまとめが答え）。
2. **URLの逆引き**：`kosu/urls.py`（§6）を開き、URL `member_new/` の担当が **どのファイルのどのクラス** か答える。さらに、そのクラスがどの `_views.py` にあるか確認する。
3. **モデルのカラム探し**：`kosu/models.py`（§8）で、「従業員が管理者かどうか」を表すカラムの **名前と型** を答える（ヒント：§8.2⑦）。
4. **設定の確認**：`hozen_another/settings.py`（§12）で、(a) 1ページに何件表示するか、(b) どのデータベースを使うか、(c) 言語は何か、を探して答える（§12.4・§12.6）。
5. **フロントとバックの名前の一致**：`Login.tsx`（§5.3⑤）が送るデータのキー名と、`main_views.py` の `Login`（§7.2⑤）が取り出すキー名が一致していることを、両方のコードで確認する。
6. **ルートと画面の対応**：`index.tsx`（§10.2）で、URL `/member-list` のとき表示される画面部品の名前を答え、その部品がどのフォルダから `import` されているかを確認する。
7. **（応用・要注意）初期管理者の確認**：`main_views.py` の `Help` クラス（§7.3）を読み、「全データを初期化したら、どの従業員番号でログインできるか」を答える（ヒント：§7.3⑦）。これは引き継ぎで知っておくべき重要事項です。

16. この章のまとめ

- **このアプリの目的** は工数（作業にかけた人の時間）の記録・集計。ログイン／工数／人員／班員／定義／問い合わせ／管理者の各機能があり、フォルダがそれに対応する（§1・§13）。
- **Webアプリには3者** がいる —— **ブラウザ**（画面）・**サーバー**（処理）・**データベース**（保存）。リクエストとレスポンスでやり取りし、HTTPメソッド（GET/POST/PUT/DELETE）で目的を区別する（§2）。
- 本アプリは **フロント（React）とバック（Django）を分離** し、**JSON** でデータをやり取りする。窓口は `/api/` で始まるURL（§3・§9）。
- **ログイン1本を6ファイルで端から端まで追った**：`Login.tsx`（入力・送信）→ `hozen_another/urls.py`（`/api/`→kosu）→ `kosu/urls.py`（`login/`→Login）→ `main_views.py`（`member` 表に問い合わせ・成功/失敗判定・セッション保存）→ JSON返却 → `Login.tsx`（`navigate("/")`）→ `index.tsx`（`/`→`MainMenu` 表示）。**他の全機能も同じ骨格**（§4〜§10）。

- `models.py` はデータの設計図。1モデル＝1テーブル、1フィールド＝1カラム。`member`（人員。`authority` / `administrator` で3段階権限）・`Business_Time_graph`（工数）・`kosu_division`（定義。1レコードに最大50区分）が中核。`id`（pk）は自動付与（§8）。
- **技術スタック** は React / TypeScript / Django / DRF / PostgreSQL を中心とした **定番で保守しやすい組み合わせ**。7基準で見ると「情報が豊富・保守しやすい・将来性が高い」のが強み（§11）。
- `settings.py` はアプリの設定ダイヤル盤。秘密情報は `.env` から読み（漏洩防止）、`INSTALLED_APPS` で `kosu` を有効化、`CORS` で別ポート通信を許可、`DATABASES` でPostgreSQL接続、`STATICFILES` / `TEMPLATES` でReactのビルド成果物を配信（§12）。
- **フォルダ構成の地図** を持っておけば、「どこを直せばよいか」で迷わない。見た目・処理は `frontend/src/`、データ・計算・権限は `kosu/`、設定は `hozen_another/`（§13）。

これで、このアプリの **全体像という地図** が手に入りました。次は、実際に自分のPCでこのアプリを動かせるようにする環境構築です。

次は、「01_環境構築.md」へ進みます。